

Multi-level Editing of Hierarchical Documents

Claudia-Lavinia Ignat^{1,*} & Moira C. Norrie²

¹LORIA INRIA Nancy-Grand Est, 54506 Vandœuvre-lès-Nancy, France (E-mail: ignatcla@loria.fr);

²ETH Zurich, 8092 Zurich, Switzerland (E-mail: norrie@inf.ethz.ch)

Abstract. Collaborative editing enables a group of people to edit documents collaboratively over a computer network. Customisation of the collaborative environment to different subcommunities of users at different points in time is an important issue. The model of the document is an important factor in achieving customisation. We have chosen a tree representation encompassing a large class of documents, such as text, XML and graphical documents and here we propose a multi-level editing approach for maintaining consistency over hierarchical-based documents. The multi-level editing approach involves logging edit operations that refer to each node. Keeping operations associated with the tree nodes to which they refer offers support for tracking user activity performed on various units of the document. This facilitates the computation of awareness information and the handling of conflicting changes referring to units of the document. Moreover, increased efficiency is obtained compared to existing approaches that use a linear structure for representing documents. The multi-level editing approach involves the recursive application of any linear merging algorithm over the document structure and we show how the approach was applied for real-time and asynchronous modes of collaboration.

Key words: consistency maintenance, operational transformation, merging of hierarchical documents, real-time collaborative editing, asynchronous collaborative editing, multi-level editing, computer-supported cooperative work

1. Introduction

Collaboration is a key requirement of teams of individuals working together towards some common goal. Computer-supported collaboration is an increasingly common occurrence, driven by the evolving global nature of business, science and engineering and enabled by improvements in computing and communication technologies. Central to collaboration is a shared information space which enables members of a community to develop together individual documents, collections of related documents or, more generally, any form of information materials relevant to their common goal.

Collaborative editing systems is a subfield of Computer-Supported Cooperative Work (CSCW) investigating ways to support a group of people editing documents collaboratively over a computer network. The collaboration between users can be synchronous or asynchronous. Synchronous collaboration means that members of

*The work presented in this paper was done when the author was member of the GlobIS group at ETH Zurich, Switzerland.

the group work at the same time on the same documents and modifications are seen in real-time by the other members of the group. Asynchronous collaboration means that members of the group modify the copies of the documents in isolation, afterwards synchronizing their copies to re-establish a common view of the data.

In collaborative writing applications such as a news agency or the authoring of scientific papers, the text document is the central unit of collaboration. Depending on the different stages and tasks of a work process of a team, synchronous or asynchronous collaboration may be required.

Consider the example of two PhD students writing a research paper together with their professor. At the beginning they decide together on the structure of the paper and divide the work of writing different sections. After writing different sections, their work is easily merged because the parts that they have been working on do not overlap. Even though they have been assigned separate parts of the document to work on, some parts of the document such as the bibliography or the introduction may be edited by all users. Moreover, at a later stage, the sections written by one of the authors will be read and modified by the other authors. At an early stage of the paper, the number of modifications performed in parallel can be maximised. In this case, setting the conflict resolution at the word level would be appropriate, i.e. conflict is detected only if modifications have been performed on the same word. But, at a later stage when changes are critical, the conflict resolution can be set at the paragraph level. This means that if two modifications have been performed in the same paragraph, the author committing the changes has to carefully read the two versions of the paragraph and decide which version to keep. In the case that the last version from the repository was committed by the professor, the students might choose to synchronise their local workspaces in accordance with the automatic policy of keeping the changes from the repository in the case of a conflict. In this way, the changes of the professor will have priority over the changes of the students.

Before a paper deadline when there is time pressure for the authors of the paper to make final edits, the real-time feature could be very helpful in order to avoid the time consuming synchronisation of the copies of the shared document and be aware of the changes performed by the other members. Different colours can be assigned to users to distinguish between their edits. Suppose the two PhD students together with their professor agreed that the professor who is a native English speaker goes through the whole paper and checks the English, while the PhD students add some new sections. The professor can go through the whole document and correct the English and then go on to revise the content and correct the English of the sections edited in the meanwhile by the two PhD students and distinguished by a different colour. Even though the real-time feature may be a necessity due to the time pressure before the deadline, the students may want to have the facility that instead of sending each character typed to the other users immediately after editing, they can write and check whole sentences or paragraphs before making the text visible to the other users involved in the collaboration.

The scenario described above shows how users may want to reveal and synchronise their activities in different ways while collaboratively authoring a document and that this may depend on the stage of the document processing as well as the tasks of the users and individual preferences. To support this, much more flexible forms of synchronisation are required than currently supported in existing collaborative editors and revision control systems. In the case of versioning systems such as CVS and Subversion, the definition of conflicts at different semantic levels such as sections and paragraphs is not possible. Similarly in current real-time text editing systems, there is no system that allows the user to work at different levels of granularity. We therefore propose an approach that allows users to work and define conflicts at different granularity levels.

Replication of data is necessary in collaborative editing to allow users to work disconnected in the case of asynchronous communication and to achieve high responsiveness in the case of real-time communication. The operational transformation approach has been identified as an appropriate approach to be used for a collaborative editing system with a replicated architecture to maintain consistency of the copies of shared documents. It has been applied to real-time collaboration (Ellis and Gibbs 1989; Ressel et al. 1996; Nichols et al. 1995; Zaffer et al. 2001; Sun et al. 1998; Suleiman et al. 1997; Sun and Ellis 1998; Vidot et al. 2000; Li and Li 2004, 2005; Sun and Sun 2006) where local operations are executed immediately after they are generated and remote operations are transformed against the other operations. The transformations are performed in such a manner that the intentions of the users are preserved and, at the end, the copies of the documents converge.

Operational transformation has also been used for the merging of documents based on operations, the so-called operation-based merging approach. Commercial versioning systems such as RCS (Tichy 1991), CVS (Berliner 1990), ClearCase (Allen et al. 1995) and Subversion (Collins-Sussman et al. 2004) use a state-based merging approach where only the information about the states of the documents and no information about the evolution of one state into another is used. On the other hand, the operation-based merging approach (Lippe and van Oosterom 1992; Shen and Sun 2002) keeps information about the evolution of one document state into another in a buffer containing a history of the operations performed between the two states of the document. The merging is done by executing the operations performed on a copy of the document on the other copy of the document to be merged. In contrast to the state-based approach, the operation-based approach does not require that the documents are transferred over the network between the local workspaces and the repository. Moreover, no complex differentiation algorithms for text such as diff (Myers 1986) have to be applied in order to compute the delta between the documents. Therefore, the responsiveness of the system is better in the operation-based approach. Merging based on operations also offers better support for conflict resolution by having the possibility of tracking user operations. In the case of operation-based

merging, when a conflict occurs, the operation causing the conflict is presented in the context in which it was originally performed. In the state-based merging approach, the conflicts are presented in the order in which they occur within the final structure of the document. In the state-based systems there is no flexible way of specifying the possible forms of conflict and merging is performed on a line by line basis with the basic unit of conflict therefore being the line. This means that the changes performed by two users are deemed to be in conflict if they refer to the same line. Concurrent changes on different lines are merged automatically. Therefore, these systems cannot handle multiple changes within a single line and semantic-based merging as presented in the above scenario is not possible.

Most of the existing collaborative editing approaches based on operation transformations (Ellis and Gibbs 1989; Ressel et al. 1996; Sun et al. 1998; Suleiman et al. 1997; Sun and Ellis 1998; Vidot et al. 2000; Li and Li 2004, 2005) adopt a linear document structure. For instance, text documents are seen as a sequence of characters. Operations target characters and, in the face of concurrent operations, syntactic consistency is achieved by ensuring the execution of all concurrent operations. Consider a shared document that contains the text: "He like the book." Assume that a user adds the letter "s" at the end of the word "like" in order to obtain "He likes the book." At the same time, another user, inserts the letter "d" at the end of the word "like" in order to obtain "He liked the book." The result obtained after the execution of the two concurrent operations is "He likesd the book." The definition and resolution of conflicts does not take into account the structure of the document, such as paragraphs, sentences and words. For instance, in the above example, a conflict could have been defined at the word level, so that two operations would be in conflict when they refer to the same word. In this way, only one of the two operations would be executed, according to the conflict policy used. For example, the policy might be that the user with the highest priority could choose which of the two conflicting operations to execute.

Some operation transformation approaches for merging have been defined for hierarchical documents such as SGML (Davis et al. 2002), XML documents (Molli et al. 2002). Even if the structure of the documents is hierarchical, the operational transformation approach is similar to the approach for linear structures and it does not take advantage of the tree structure of the document. The existing operation-based approaches maintain a single history buffer of the operations executed. Operations are not associated with the structure of the document and therefore it is difficult to select which operations refer to which node in the document. Moreover, the above mentioned approaches adopt only automatic resolution of conflicts where the effect of all operations is maintained and they do not allow for the flexible definition and resolution of conflicts. For instance, they do not allow the possibility of defining that all operations referring to the same node are in conflict and allowing the user to later choose one of the versions of the node. To determine which operations from the history buffer refer

to which node is very complex, since the structure of the document is dynamically changed with the execution of each operation.

Multi-level editing involves logging edit operations that refer to each node. In this way, conflicting operations that refer to the same subtree of the document are easily detected by the analysis of the histories associated with the nodes belonging to the subtree. Therefore, the resolution of conflicts is simplified in comparison to the approaches that keep operations in a single history buffer. Moreover, conflict levels can be dynamically varied and conflict units can be presented in the context in which they occurred or at a higher level. For instance, if conflict was defined at the level of a particular element in a document, meaning that two operations changing that element are in conflict, the conflict can be presented at the level of that element or at the level of one of the ancestor elements.

Keeping the operations that refer to a node associated with that node also offers support for tracking user activity performed on that node. For instance, this could be helpful for providing awareness information regarding the changes that have been made to that node (Papadopoulou et al. 2006).

By using multi-level editing, support for concurrency is increased. Two operations are considered in conflict only if they target a common node in the tree. In the approaches where operations are kept in a single buffer, when a new operation has to be integrated into the history buffer, the entire history has to be scanned and transformations need to be performed even though changes refer to completely different elements in the document and do not interfere with each other. In the multi-level editing approach, the number of transformations that have to be performed is significantly reduced as operations belonging to two nodes that are on different branches of the tree are commutative and they do not need transformations.

In this paper, we propose a multi-level editing approach that maintains the consistency of text documents both for the real-time and asynchronous modes of collaboration. The structured model of the document enables flexible granularity for the propagation of changes over the network, for the detection and resolution of conflicts and for details about user activity on different parts of the document. Specifically, we model a text document as a hierarchical structure consisting of paragraphs, sentences, words and characters. We show how the multi-level editing approach can be applied to both synchronous and asynchronous modes of collaboration. Our approach is based on the operational transformation mechanism and achieves better efficiency compared to existing approaches for merging documents with linear structures either in real-time or asynchronously. While we present our approach applied to text documents, it should be noted that the algorithms are general and can be adapted for any type of document conforming to a hierarchical structure.

The basic ideas of our multi-level editing approach for consistency maintenance in real-time collaboration based on a tree representation of documents was previously presented in a conference paper (Ignat and Norrie 2003). The current paper is an extensive revision of this publication describing the application of the approach not only for the real-time communication, but also for asynchronous

collaboration. We additionally provide a complexity analysis of our approaches and discuss applications of the multi-level editing approach for providing awareness and handling conflicts.

We start in the next section by introducing the basic notions of the operational transformation mechanism and by presenting the existing operational transformation approaches for real-time and asynchronous collaborative editing. We then go on in section 3 to present how we modelled the document subject to collaboration and the operations exchanged during the collaboration process. In section 4, we describe our treeOPT approach for maintaining consistency in real-time collaboration over hierarchical documents. In section 5, we present the asyncTreeOPT approach which is an adaptation of our treeOPT algorithm to handle merging in asynchronous collaboration. Besides the principles of our basic merging approach, we also discuss some other issues involved in merging such as the transformation functions used in the merging approach and the compression of the log before merging is performed. In section 6, we describe the application of the multi-level editing approach for providing awareness information and for handling conflicts referring to a certain part of the document. In section 7, we analyse the complexities of our multi-level editing approaches for both real-time and asynchronous modes of communication and show the efficiency of our mechanism with respect to other approaches that use a linear history of operations. In section 8, we discuss the problems encountered in splitting and joining elements of the hierarchical structure and the solutions that we adopted. In section 9, we compare our approach with existing approaches for maintaining consistency in real-time and asynchronous collaborative editing. Concluding remarks and the directions of our future work are presented in the last section.

2. Operational transformation approaches

In this section, we first present the basic notions of the operational transformation approach and then analyse some of the existing operational transformation approaches for real-time and asynchronous collaboration.

2.1. Basic notions

The main assumption of the OT algorithms is that each site involved in the collaboration maintains a local copy of the shared text document and the aim of these algorithms is to maintain consistency among these copies. Most of the existing algorithms assume a linear structure of the text document, i.e. a sequence of characters. The operations that can be performed are

- $insert(p, c)$ – inserts the character c at the position p
- $delete(p)$ – deletes the character at position p

Some of the existing approaches consider that the position of the first character in the document is 0, while other algorithms consider that the position of the first character is 1. For uniformity reasons, throughout this paper we are going to present all algorithms by considering that the position of the first character is 1.

Figure 1 illustrates a very simple example of the operational transformation mechanism. Suppose the shared document contains the following text “concurrency contrl”. Two users, at $Site_1$ and $Site_2$, respectively, concurrently perform some operations on their local replicas of the document. $User_1$ performs operation O_1 of inserting the character ‘r’ at position 7 in order to correct the misspelling of the word “concurrency” to obtain “concurrency contrl”. Concurrently, $User_2$ performs operation O_2 of inserting the character ‘o’ at position 17, in order to correct the misspelling of the word “contrl” to obtain “concurrency control”.

At $Site_1$, when operation O_2 arrives, it needs to be transformed against operation O_1 to include the effect of this operation. Due to the fact that the operation O_1 executed concurrently with O_2 inserted a character before the character to be inserted by operation O_2 , operation O_2 needs to adapt the position of insertion, i.e. increase its position by 1. In this way the transformed operation O_2 will become an insert operation of the character ‘o’ into position 18, the result being “concurrency control”, satisfying the intentions of both users. At $Site_2$, in the same way, operation O_1 needs to be transformed against O_2 in order to include the effect of O_2 . The position of insertion of O_1 does not need to be modified in this case because operation O_2 inserted a character to the right of the insertion position of O_1 . Therefore, the transformed operation O_1 has the same form as the original operation O_1 . We see that the result obtained at $Site_2$ respects the intentions of the two users and, moreover, the two replicas at the two sites converge. This form of transformation is called *inclusion transformation* (Sun et al. 1998; Sun and Ellis 1998; Li and Li 2004).

Between operations performed by users there exist *precedence* relations that have to be maintained. Most of the consistency maintenance algorithms that use the operational transformation mechanism such as (Ellis and Gibbs 1989; Sun and Ellis 1998; Suleiman et al. 1997) define the precedence relation based on the “happened before” relation of Lamport (Lamport 1978). An operation O_1 is said to causally *precede* O_2 denoted by $O_1 \rightarrow O_2$ if O_2 was generated on a state where

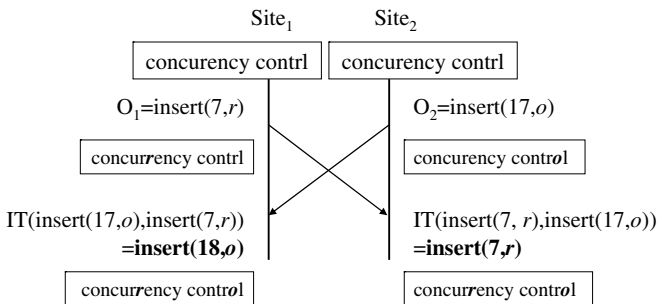


Figure 1. Scenario illustrating the operational transformation mechanism.

O_1 was previously executed. Two operations O_1 and O_2 are said to be concurrent, denoted by $O_1 \parallel O_2$ iff neither $O_1 \rightarrow O_2$, nor $O_2 \rightarrow O_1$. For instance, in Figure 2, $O_1 \parallel O_2$, $O_1 \parallel O_3$ and $O_2 \rightarrow O_3$.

In what follows, we define the notion of operation context, as well as the notions of contextual equivalence and contextual precedence between operations, used by several operational transformation algorithms. The *state* of a document is associated with the list of operations that have to be executed to bring the document from its initial state to the current state. The *context* (Shen and Sun 2002) of an operation O is the document state on which O 's parameters are defined. Two operations O_a and O_b are *context equivalent* (Shen and Sun 2002) denoted by $O_a \sqcup O_b$ if the contexts of these operations are equal. Given two operations O_a and O_b , O_a is *context preceding* (Shen and Sun 2002) O_b denoted by $O_a \mapsto O_b$ if the state resulting after the execution of O_a is the context of O_b .

2.2. OT approaches for linear-structured documents

In this section, we present some of the existing operational transformation approaches working for linear structures that we used in our approach by applying them recursively over hierarchically structured documents. We first describe the linear operational transformation approaches for real-time collaboration and afterwards those for asynchronous collaboration.

2.2.1. Approaches for real-time collaboration

The work that pioneered the use of operational transformation in the field of collaborative editing was the dOPT (Ellis and Gibbs 1989) approach. However, this algorithm did not provide a solution for the case that transformations have to be performed between two operations that do not have the same context.

The approaches that followed dOPT, such as SOCT2 (Suleiman et al. 1997, 1998), GOT (Sun et al. 1998) and GOTO (Sun and Ellis 1998) identified the condition of applying the transformation of operation O_1 against O_2 as being that operations O_1

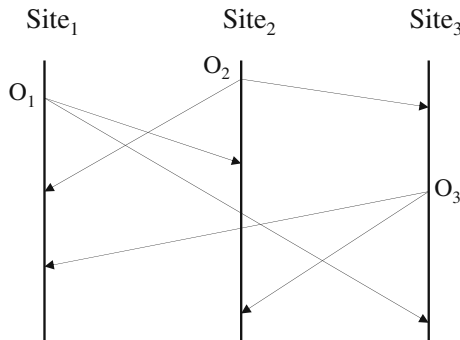


Figure 2. Concurrent and precedent operations: $O_1 \parallel O_2$, $O_1 \parallel O_3$, $O_2 \rightarrow O_3$.

and O_2 should have the same contexts denoted by $O_1 \sqcup O_2$. These approaches found a solution for the unresolved problem described in (Ellis and Gibbs 1989) and illustrated in Figure 2. The relations between the operations are the following $O_1 \parallel O_2$, $O_2 \rightarrow O_3$ and $O_1 \parallel O_3$. Let us analyse how the operations are executed at $Site_1$. When operation O_2 arrives at $Site_1$, it is transformed against O_1 in order to include the effect of operation O_1 . The function that transforms O_2 against O_1 is called inclusion transformation and is denoted by $IT(O_2, O_1)$ (Sun et al. 1998). O_2 can be transformed against O_1 as both operations have the same context. But, when operation O_3 arrives at the site, it cannot be forward transposed against O_1 because O_1 and O_3 do not have the same context. The context of operation O_3 contains operation O_2 , while operation O_1 does not. In order to respect the conditions for performing transformations, in the GOT and GOTO (Sun et al. 1998; Sun and Ellis 1998) approaches, another function called exclusion transformation was defined while the SOCT2 (Suleiman et al. 1997) approach defined the backward transposition function. The *exclusion transformation* $ET(O_a, O_b)$ returns the form of the operation O_a that excludes the preceding operation O_b from its context. The condition of applying the exclusion transformation is that the state resulting after the execution of O_b is the context of O_a , i.e. $O_b \mapsto O_a$. In the previous example, O_3 would have to exclude the effect of O_2 before being transformed against O_1 . The backward transposition function defined in SOCT2 changes the execution order of two operations and transforms them such that the same effect is obtained as if the operations were executed in their initial order and initial form. In the previous example, a backward transposition between O_1 and the transformed form of O_2 against O_1 , i.e. O'_2 , is performed. The backward transposition function computes the form of operation O'_2 as if O_1 had not been executed, the result being O_2 , and it computes the form of operation O'_1 , i.e. O'_1 , that includes the effect of O_2 . In this way, O_3 can be transformed against O'_1 as both have the same context, i.e. they both include operation O_2 in their contexts.

The principle of both the SOCT2 and GOTO algorithms is that the executed operations are kept in a history buffer and when a remote operation arrives at the site, if it is causally ready, it has to be integrated in the history buffer. An operation is said to be causally ready if all operations preceding it have been integrated at that site. In order to integrate the remote operation, the whole history buffer has to be traversed and reordered such that all operations that are causally preceding the remote operation come before the operations that are concurrent to the remote operation in the history buffer. Afterwards, the remote operation has to be transformed according to the sequence of concurrent operations.

The GOT (Sun et al. 1998) algorithm defines a total order between operations in order to ensure convergence in the presence of concurrent operations. Based on the total order, an undo/do/redo scheme has been defined. When a remote causally ready operation arrives at a site, all the operations in the history buffer that follow it in the total order have to be undone in order to restore the document to the state before their execution. Afterwards the remote operation is transformed and executed and the operations that have been undone are in turn transformed and re-executed.

In the adOPTed (Ressel et al. 1996), Jupiter (Nichols et al. 1995) and NetEdit (Zaffer et al. 2001) approaches, directed graphs are used instead of history buffers to model the interactions between users involved in the collaboration. The vertices of the graphs represent the application states, while the edges of the graph represent the original user requests or the result of operation transformations. In these approaches, in order to compute the execution form of the remote operation, the computation of the intermediate states is required.

SOCT3 and SOCT4 algorithms (Vidot et al. 2000) are also approaches for real-time communication that use the operational transformation mechanism, but they rely on a form of centralization and therefore are not suited to peer-to-peer networks. In both algorithms, the operations are globally ordered according to timestamps generated by a sequencer.

In (Li and Li 2004), the authors propose an approach based on the effects relation between operations. The effects relation between operations relies on the fact that there exists a total order relation between the target characters of the operations. The effects relation between operations is used as a criteria for distinguishing the different cases for the inclusion and exclusion transformation functions instead of the current positions of operations. The last synchronisation point between two operations, i.e. the latest common state on which the operations are defined, and the relations between the operations according to the last synchronisation point are computed. The control algorithm used in the effects relation approach is similar to the control algorithm in the SOCT2 approach.

All of the operational transformation algorithms described above keep a single history of operations already executed in order to compute the proper execution form of new operations. When a new remote operation is received, the whole history needs to be scanned and transformations need to be performed, even though different users might work on completely different sections of the document and do not interfere with each other. Keeping the history of all operations in a single buffer decreases the efficiency.

As shown in section 7, the complexity of existing algorithms such as GOT, GOTO and SOCT2 for integrating a new causally ready operation into the history is of order n^2 , where n is the size of the examined history buffer. Exceptionally, the dOPT algorithm has a complexity of order n , but convergence of copies is not always achieved. Consequently, a long history results in a higher complexity. This complexity negatively affects responsiveness, which is a factor of critical importance in real-time editing systems.

2.2.2. Approaches for asynchronous collaboration

Most configuration management tools support the copy/modify/merge paradigm. It consists basically of three methods applied on a shared repository storing multiversioned objects: checkout, commit and update. A *checkout* method creates a local working copy of an object from the repository. A *commit* method creates a new version of the corresponding object in the repository by validating the

modifications done on the local copy of the object. The condition of performing this method is that the repository does not contain a more recent version of the object to be committed than the local copy of the object. An *update* method performs the merging of the local copy of the object with the last version of that object stored in the repository.

We will now describe the FORCE (Shen and Sun 2002) approach for merging that is used in our merging approach by recursively applying the FORCE algorithm over the document structure.

In the commit phase of merging, a check is first performed as to whether the user can commit the changes to the repository. If the base version of the document in the local workspace, i.e. the last version from the repository that the user started working on, is equal to the last version in the repository, a commit can be performed. Otherwise, an update is necessary before committing the data. In the case that a commit is allowed to be performed, the repository simply executes sequentially the operations that were performed in the local workspace in order to generate the full state of the latest version from the repository. The previous version from the repository is replaced with the received operations from the local workspace, representing the delta between the new version and the previous version. Additionally, the corresponding base version number from the local workspace as well as the latest version number from the repository are increased and the local log from the local workspace is emptied.

In the checkout phase, a request is sent to the repository including the version number of the document that is intended to be checked out. In the case that the requested version number is larger than the latest version number in the repository, the repository sends a reject reply. In the case that the requested version number equals the latest version number in the repository, the repository sends the full state of the last version of the document to the local workspace. In the case that the requested version number is less than the latest version number from the repository, the repository generates the state of the requested version by executing the inverses of the operations representing the deltas between the latest version in the repository and the requested version. In the case of a positive reply from the repository, the local site sets the received document as the working copy and assigns the version number of the document that was sent to the base version number.

In the updating phase, the site sends the number of the base version to the repository. The repository sends to the site a list of operations representing the delta between the latest version in the repository and the base version. Upon receiving the list of operations from the repository, the local workspace performs the merging algorithm and updates the base version number. The merging scenario is illustrated in Figure 3.

The local user started working from version V_k on the repository but cannot commit their changes because meanwhile the version from the repository has been updated to version V_{k+n} . Let us denote by LL the list of operations executed

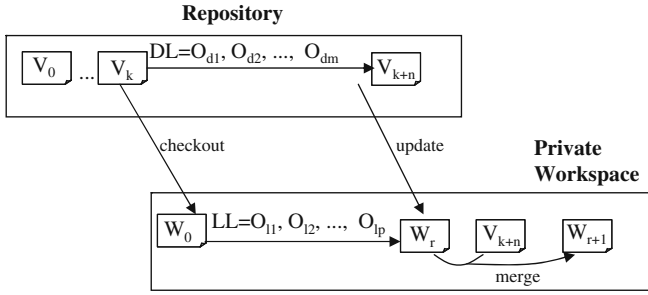


Figure 3. Updating stage of the merging.

by the user in their local workspace and by DL the list of operations representing the delta between versions V_{k+n} and V_k . Two basic steps have to be performed. The first step consists of applying the operations from DL on the local copy of the user in order to update the local document to version V_{k+n} . The operations from the repository, however, cannot be executed in their original form, as they have to be transformed in order to include the effect of all local operations before they can be executed in the user workspace. The second step consists of transforming the operations in LL in order to include the effects of the operations in DL . The resulting list of transformed local operations represents the new delta in the repository.

FORCE adopts an additional abstraction layer which allows a complete separation of the syntactic merging from the semantic merging by means of the semantic conflict function. A semantic merging policy is specified as a set of semantic merging rules and a function *semanticConflict* determines whether two concurrent operations are semantically conflicting. From the list of operations in the list DL not all of them can be executed in the local workspace because some of these operations may be in conflict with some of the operations from LL . Let us consider that $DL = [O_{d1}, \dots, O_{d(i-1)}, O_{di}, O_{d(i+1)}, \dots, O_{dm}]$, where $O_{d1} \mapsto \dots \mapsto O_{dm}$. In the case that O_{di} is in conflict with at least one operation from LL , O_{di} cannot be executed in the local workspace. Moreover, all operations following it in the list DL need to exclude its effect from their context. But, as we have already seen, the condition to exclude an operation O_a from an operation O_b is that $O_a \mapsto O_b$. Therefore, in order to exclude the effect of operation O_{di} from the context of all operations following it in the list DL , operation O_{di} needs to be transposed towards the end of the list DL . The transposition between two operations changes the execution order of those operations and transforms them such that the same effect is obtained as if the operations were executed in their initial order and initial form.

In order to combine the two steps of merging, i.e. the computation of the transformations of the operations from the repository against the operations from the local log and the transformations of the operations from the local log against

the repository, symmetric inclusions between pairs of operations from the local and remote logs are computed (Shen and Sun 2002).

The basic *merge* procedure takes as arguments the remote log, the local log and the base version number at the local site. It generates as output two other logs, the new remote log and the new local log, each of which is modified to include the effects of the operations in the other log. The new remote log will contain the list of operations that should be executed sequentially on the current document state of the working copy in order to update it. It will contain the non conflicting operations from the original remote log, modified in order to include the effects of the operations in the local log. The new local log will store the list of operations which represent the delta between the new version and the old version in the repository and will have to be sent to the repository. It contains the operations in the local log transformed in order to include the effect of the operations in the remote log. Additionally, it might also include the inverse of the conflicting operations from the remote log.

3. Document model

We model a text document as a hierarchical structure having the following levels of granularity: document (0), paragraph (1), sentence (2), word (3) and character (4), document being the highest granularity level and character being the lowest granularity level. For real-time collaboration, each site maintains a copy of the shared hierarchical document. In the case of asynchronous communication, each workspace stores locally a copy of the tree structure of the document. The hierarchical structure is created when the document is checked out from the

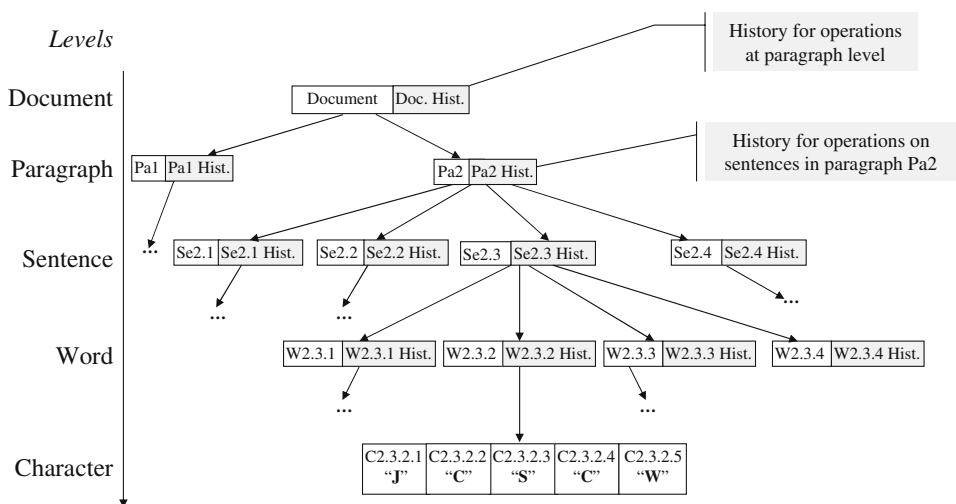


Figure 4. Document structure.

repository and is modified while changes are performed locally. Each node in the hierarchical model (excluding leaf nodes) will keep a history of insertion and deletion operations associated with its child nodes. The structure of a document is illustrated in Figure 4.

DEFINITION 1. *A node N of a document is a structure of the form $N = \langle \text{parent}, \text{children}, \text{length}, \text{history}, \text{content} \rangle$, where*

- *parent is the parent node for the current node. Except for the topmost node, parent is a valid reference to a node in the tree.*
- *children is an ordered list $[\text{child}_1, \dots, \text{child}_n]$ of child nodes*
- *length is the length of the node in terms of the number of leaf nodes*

$$\text{length} = \begin{cases} 1, & \text{if } N \text{ is a leaf node} \\ \sum_{i=1}^n \text{length}(\text{child}_i), & \text{otherwise.} \end{cases}$$

- *history is an ordered list of operations executed on child nodes*
- *content is the content of the node, defined only for leaf nodes*

$$\text{content} = \begin{cases} a\text{Character}, & \text{if } N \text{ is a leaf node} \\ \sum_{i=1}^n \text{content}(\text{child}_i), & \text{otherwise.} \end{cases}$$

The level of a node is the height of the node, i.e. the length of the path from the root to the node.

By using a structured model for the representation of documents, support for collaboration is offered for a large class of documents. Besides text documents, XML, which has become a popular format for marking up various kinds of data from web content to application data, conforms by definition to a hierarchical structure. Source code documents written in an object-oriented programming language also conform to a tree structure: a document contains a set of classes, each class contains a set of fields and methods and each method contains a set of lines of code.

The operations exchanged between sites during the collaboration refer to parts of the hierarchical structure of the text document corresponding to different levels of the document: paragraph, sentence, word and character. We now present the definition of the operations used in our tree model. We call them composite operations in order to distinguish them from regular operations defined for linear document structures.

DEFINITION 2. *A composite operation is a structure of the form $cOp = \langle \text{type}, \text{position}, \text{content}, \text{stateVector}, \text{initiator} \rangle$, where*

- *type is the type of the operation*
- *position is a vector of positions specifying the path starting from the root to the node where the operation is applied*

- *content is a node representing the content of the operation*
- *stateVector is the state vector of the generating site*
- *initiator is the identifier of the site that generated the operation*

The level of an operation is the level of the node in whose history the operation is kept. For instance, in a text editing application where the levels of the document are the document (level 0), the paragraph (level 1), the sentence (level 2), the word (level 3) and the character (level 4), an *insertParagraph* operation belongs to the document history and is of level 0, an *insertSentence* operation is of level 1, an *insertWord* operation is of level 2 and an *insertChar* operation is of level 3.

In a simple text editing application where a document is composed of paragraphs, sentences, words and characters, two types of operations have been used: insertion and deletion. A vector position associated with an operation specifies the positions for the levels corresponding to a coarser or equal granularity than the granularity of the operation. For example, if we have an insertion operation of word, we have to specify the paragraph and sentence in which the word is located, as well as the position of the word within the sentence. The content of a composite insertion operation specifies the node to be inserted in the position given by the position vector. For the sake of simplicity, in future examples, we will denote operations by specifying only their type, level, position and the text conversion of content, ignoring the other attributes. For example, *insertWord* (2,3,2, "JCSCW") denotes a composite operation of type insertion, having the level word, in paragraph 2, sentence 3, at word position 2 inside the sentence, and having as content a node of type word which stands for the text "JCSCW".

4. treeOPT for real-time collaboration

In this section, we present our treeOPT approach for maintaining consistency in real-time collaboration.

We first give an intuitive explanation of the algorithm, and afterwards describe it formally. As already mentioned, each site stores locally a copy of the hierarchical structure of the shared document. The leaf nodes are character nodes. The content of a leaf node is explicitly specified in the content field. The content field of an internal node remains unspecified, but the actual content of the node is computed as the concatenation of the contents of its children. Each node (excluding leaf nodes) will keep a history of insertion and deletion operations associated with its child nodes. An example showing the structure of a document is illustrated in Figure 4, where the document contains two paragraphs; paragraph 2 contains four sentences; sentence 3 of paragraph 2 contains four words; the second word of sentence 3 in paragraph 2 is "JCSCW".

The principles of the algorithm are described in what follows. Each site can generate composite operations, representing insertions or deletions of subtrees in

the document tree. Note that each node of a subtree to be inserted has an empty history buffer. The site generating a composite operation executes it immediately. The operation is also recorded in the history buffer associated to the parent node of the inserted or deleted subtree. Finally, the new operation is broadcast to all other sites, being timestamped using a state vector. Upon receiving a remote operation, a site will test it for causal readiness. If the composite operation is not causally ready it will be queued, otherwise it will be transformed and then executed.

We illustrate how transformations are performed using an example. Consider a site receiving the following remote composite operation: *insertWord*(2,3,2, "JCSCW"). It is an operation intending to insert the word "JCSCW" in paragraph 2, sentence 3, as the second word. The newly received operation must be transformed against the previous operations, as described below. First of all, we consider the paragraph number specified by the composite operation, which in this case is equal to 2. We do not know for sure that paragraph number 2 of this site's local copy of the document is the same paragraph as that referred to by the original operation. Suppose a concurrent operation inserted a whole new paragraph before paragraph 2. Then, in this case, we should insert the word "JCSCW" not in paragraph 2, but in paragraph 3. Therefore, we must first transform the new operation against previous operations involving whole paragraphs, which are kept in the document history buffer. This can be done using any existing operational transformation algorithm working on linear structures such as the GOT(O) or SOCT2 algorithms. After performing these transformations, we obtain the position of the paragraph in which the operation has to be performed, paragraph number 3 in our example. Consequently, the new composite operation becomes *insertWord*(3,3,2, "JCSCW"). Here it is important to note that operations of finer granularity are not taken into account by these transformations as the document history buffer contains only operations at the paragraph level. Indeed, we are not interested in whether another user has modified another paragraph, because this fact does not affect the number of the paragraph where the word "JCSCW" has to be inserted. The next step obtains the correct number of the sentence where the word has to be inserted. Therefore, the new operation is transformed against the operations belonging to *Pa3 history*. *Pa3 history* only contains insertions and deletions of sentences that are children of paragraph 3. We again apply an existing operational transformation algorithm, and obtain the correct sentence position (for example sentence 4), transforming the operation into *insertWord*(3,4,2, "JCSCW"). The algorithm continues by obtaining the correct word position in the same manner. Finally, the operation can be executed and recorded in the history. Because it is an operation of word level, it must be recorded in the history associated with the parent sentence. As we can see, the algorithm achieves consistency by repeatedly applying an existing concurrency control algorithm on small portions of the entire history of operations, which, rather than being kept in a single linear

structure, is distributed throughout the tree. We now present the general form of the treeOPT algorithm.

```

Algorithm treeOPT( $O, RN, L$ ): $O$  {
   $CN := RN$ ;
  for ( $l := 1; l \leq L; l++$ ) {
     $O_{new} := composite2Simple(O, l)$ ;
     $EO_{new} := transform(O_{new}, history(CN))$ ;
     $position(O)[l] := position(EO_{new})$ ;
    if ( $level(O) = l$ )
      return  $O$ ;
     $CN := child_i(CN)$ , where  $i = position(EO_{new})$ ;
  }
}
```

Given a new causally ready composite operation, O , the root node of the hierarchical representation of the local copy of the document, RN , and the number of levels in the hierarchical structure of the document, L , the execution form of O is returned. In the case of the text editor, $L = 4$ and $RN = document$. As we saw in the previous example, determining the execution form of a composite operation requires computing the elements of the position vector corresponding to a coarser or equal granularity level than that of the composite operation. For each level of granularity l , starting with paragraph level and ending with the level of the composite operation, an existing operational transformation algorithm is applied to find the execution form of the corresponding regular operation. Traditional algorithms do not perform transformations on composite operations, but rather on regular ones. Therefore, we had to define the function *composite2Simple*, that takes as arguments a composite operation together with the granularity level at which the operation is transformed and returns the corresponding regular operation. The operational transformation algorithm is applied to the history of the current node CN whose granularity level is $l - 1$. Recall that, for example, to find the corresponding paragraph position, transformations need to be performed against the operations kept in the document history. The l th element in the position vector will be equal to the position of the execution form of the regular operation. If the current granularity level l is equal to the level of the composite operation, the algorithm returns the execution form of the composite operation. Otherwise, processing continues with the next finer granularity level, with CN being updated accordingly. By *transform*(O, HB) we denote any existing concurrency control algorithm that, taking as parameters a causally-ready regular operation O and a history buffer HB , returns the execution form of O . The implementation of the *transform* method depends on the chosen consistency maintenance algorithm working on a linear structure of the document. We tested the operation of our algorithm when combined with the SOCT2 and GOT algorithm. A detailed

implementation of the treeOPT-GOT algorithm as well as of the *composite2Simple* function can be found in (Ignat and Norrie 2002).

The treeOPT algorithm is a general algorithm in that it can be applied to any document having a hierarchical structure. A trivial application would be the case of a book modelled as being composed of chapters, with each chapter consisting of sections, each section of paragraphs, each paragraph of sentences and so on. We applied the treeOPT algorithm for the case of XML documents (Ignat and Norrie 2006a; Ignat and Oster 2007), where each XML node is an element in the tree and its attributes and child elements are children of that node.

We have to mention that due to the tree structure of the document, if a user deletes an element of a higher granularity level, concurrent changes performed by the other users targeting subnodes in the deleted tree will not be executed. This is the right decision to be taken from a semantic point of view, since the existence of a node of a lower granularity level does not make sense if one of its ancestor nodes is deleted. Consider, for instance, that a user is deleting a sentence, while concurrently another user is inserting a word in that sentence. From a syntactic point of view, respecting the intentions of both users would result in the sentence deletion followed by the insertion of the word in the empty sentence. But, semantically this is not a correct result, since a word in an empty sentence loses its meaning. In the same way, an XML document would not conform to its DTD if a subelement is kept while the parent element is deleted.

An important advantage of the algorithm is related to its improved efficiency compared to the operational transformation approaches that store operations in a single buffer. In our representation of the document, the history of operations is not kept in a single buffer, but rather distributed throughout the whole tree, and, when a new operation is transformed, only the history distributed on a single path of the tree will be spanned. This will turn out to be a very important increase in speed, especially given the fact that the complexity of the concurrency control algorithms based on a single buffer is usually of $O(n^2)$ where n is the length of the spanned history, as will be shown in section 7.1. Moreover, when working on medium or large documents, operations will be localized in the areas currently modified by each individual user and these may often be non-overlapping. In these cases, almost no transformations are needed and therefore responsiveness is very good. (Recall the fact that in the case of algorithms storing operations in a single buffer, every operation interferes with any other independently of the distance between the positions specified in the operations). Another important advantage is the possibility of performing, not only operations on characters, but also on other semantic units - words, sentences and paragraphs. The transformation functions used in the operational transformation mechanism are kept simple as in the case of character-wise transformations, not having the complexity of string-wise transformations as presented in (Sun et al. 1998). An insertion or deletion of a whole paragraph can be done in a single operation. Therefore, the efficiency is further increased, because there are fewer operations to be transformed, and fewer to be transformed against. Moreover, the data is sent using larger chunks, thus the network communication is more efficient. Our approach also adds flexibility in

using the editor, since users are able to select the level of granularity that they prefer to work on.

5. **asyncTreeOPT for asynchronous collaboration**

In this section, we present how our multi-level editing approach was applied to maintain consistency over hierarchical documents in asynchronous collaboration with a central repository. We show the generality of our treeOPT algorithm by presenting how it recursively applies a linear merging algorithm for asynchronous communication over the document levels. The algorithm obtained as result of the adaptation of treeOPT for the asynchronous communication is called asyncTreeOPT.

5.1. The asyncTreeOPT merging algorithm

As we have seen in section 2.2.2, the main issue in asynchronous communication is how to deal with conflicts and cancellation of operations. Therefore, algorithms used in real-time communication have to be adapted for asynchronous communication by the integration of undo mechanisms that offer support for cancellation of operations. As will be shown in section 7.2.1, integration of an undo mechanism into a real-time operational transformation algorithm can be optimised. FORCE was proposed to adapt and optimise a real-time algorithm for asynchronous communication. For the same reason, we adapted treeOPT for asynchronous communication by the recursive application of FORCE. In this way, we obtain an optimised approach for asynchronous communication supporting hierarchical documents.

We start by describing how the basic methods of commit, checkout and update of the Commit/Modify/Merge paradigm are applied to hierarchical document structures.

The *commit* phase applied to the tree document representation follows the same principles as in the case of linear representation. The hierarchical representation of the document history is linearised using a breadth-first tree traversal. In this way, the first operations in the log will be the ones belonging to paragraph logs, followed by the operations belonging to sentence logs and finally the operations belonging to word logs.

In the *checkout* phase, the local workspace is emptied and all operations from the repository representing the delta between the version of the document the user wants to work on and the initial version of the document are executed in the local user workspace. The checkout phase could also be implemented in the same way as in the FORCE approach, as described in section 2.2.2. The main difference is that, in the FORCE approach, the latest version in the repository is the state of the document and the previous versions are represented by a list of operations constituting the delta between the versions. However, in our approach, all versions are represented by the delta list of operations and only the first version in the repository contains the state of the document.

The *update* procedure achieves the actual update of the local version of the hierarchical document with the changes that have been committed by other users to the repository and kept in the remote log. The remote log contains a linearisation of

the logs that were initially part of the document tree structure. The goal of the update procedure is the same as that of the *merge* procedure of FORCE generalised for the level of the entire document tree. It aims to replace the local log associated with each node with a new one which includes the effects of all non-conflicting operations from the remote log and to execute a modified version of the remote log on the local version of the document in order to update it to the version on the repository. The update procedure is presented in what follows.

```

Algorithm update(CN, RL) {
1:   LLL := log(CN);
      bInd := |RL|;
      RLL := [];
      for (i := 0; i < |RL|; i++) {
          O := RL[i];
          if (level(O) = level(CN))
              append(O, RLL);
          else {
              bInd := i;
              break;
          }
      }
2:   updateOpInds(LLL, indices(CN));
3:   (NRL, NLL) := merge(RLL, LLL);
4:   for (i := 0; i < |NRL|; i++)
       applyOperation(NRL[i]);
       setLog(CN, NLL);
5:   ChildRL := [];
       for (i := 0; i < noChildren(CN); i++)
           ChildRL[i] := [];
       for (i := bInd; i < |RL|; i++) {
           O := RL[i];
           for (j := 0; j < |NLL|; j++)
               include(O, NLL[j]);
               append(O, ChildRL[index(O, level(CN))]);
           }
       for (i := 0; i < noChildren(CN); i++)
           update(childAt(CN, i), ChildRL[i]);
}

```

The *CN* argument of the *update* procedure represents the current node in the tree traversal. In the initial call of the procedure, the current node is equal to the

root of the document tree. The parameter *RL* represents the remote log. Block 1 of the algorithm represents the initialisation phase. *LLL* is the local level log and *RLL* is the remote level log associated to the current node. The *RLL* is initialised with the remote operations pertaining to the current node, by iterating over the remote log and keeping those operations whose level is identical to the level of the current node. Recall that the level of an operation is equal to the level of the node in whose history the operation is kept. The *bInd* variable stores the index of the first operation that refers to a lower level than the level of the current node.

Block 2 of the algorithm includes the update of the indices of all the operations in the *LLL* so that they correspond to the current position in the tree of the node to whose log they belong. During the update algorithm, as the modified remote operations are applied on the local version of the tree, nodes might get inserted or deleted from the tree. As the positions of the nodes change, it is clear that all operations belonging to the log of the nodes whose positions have changed will no longer have valid indices. For example, if the local level log contains the operation *deleteChar*("d",1,3,4,5) and paragraph 1 has been shifted two positions to the right by the insertion of two new paragraphs before it, the operation has to be transformed to *deleteChar*("d",3,3,4,5).

In block 3 of the algorithm, the basic merge algorithm is called in order to merge the *RLL* and the *LLL* and generate two new logs, *NRL* and *NLL*. In the version of the update procedure presented in this paper, due to the merge procedure used by FORCE approach (Shen and Sun 2002), the conflict resolution policy is that the local version of the operation is the one that is kept in the case of conflicts. In the current implementation of the asynchronous text editing system, other policies for merging have also been implemented, as described in section 6.2. We mention that for our merging algorithm we can use any existing linear approach for the merging of two lists of operations. However, in our current implementation, we have used the FORCE merging algorithm.

Afterwards, in block 4 of the algorithm, the operations from the *NRL* are applied to the local copy of the document in order to update it and the local log of the current node is then replaced with the *NLL*.

In block 5 of the *update* procedure the remaining part of the remote log, i.e. the part including the operations following *bInd*, needs to be divided among the children of the current node and the update method called recursively for each child. Each operation in the remote log starting from position *bInd* will be transformed in order to include the effects of all the operations in the *NLL*. This is necessary as operations in the new local log are of higher level than the ones remaining in the remote log and thus can influence the context of the remote operations. Afterwards, the transformed remote operations will be appended to the corresponding *ChildRL* lists associated to the children of *CN*. A transformed operation is appended to *ChildRL*[*i*] if the operation's modified index corresponding to the level of the current node is equal to *i*. *index(O,L)* returns the index of operation *O* corresponding to the level *L*. By the end of the iteration,

all remote operations will have been transformed and placed in the correct list. Finally, the update method is recursively called for each child of *CN* with each of the previously created lists of operations as remote logs.

5.2. Transformation functions

We now explain how transformation functions for linear structures can be adapted for use in our approach. As shown in the *update* procedure, when merging is performed at the level of the current node of granularity *i*, the local level log *LLL* contains operations of granularity *i* and the remote log contains operations of granularity *i* composing the remote level log *RLL* and other operations that refer to the current node but of a finer granularity than *i*.

Operations from *LLL* and *RLL* are of the same level of granularity *i*, so the transformations of the operations from *LLL* against the operations from *RLL*, and conversely, modify the *i*th index of the position vector of the operations.

Each composite operation from *LLL* and *RLL* can be transformed into a simple operation, the position parameter of a simple operation representing the *i*th index of the position vector of the corresponding composite operation. Inclusion and exclusion transformation functions for simple operations can then be applied to compute the transformed positions of these operations. The transformed position of a simple operation will represent the *i*th index of the position vector of the transformed form of the corresponding composite operation.

The operations in *RL* that follow the operations in *RLL* are of a finer granularity than *i* and they have to include the effect of the operations in the new local log *NLL*. The operations in *NLL* are of granularity *i* and they might affect only the position of the operations in *RL* corresponding to level *i*. The operations in *RL* can be transformed into simple operations corresponding to level *i*. Transformation functions working for linear structures can be then applied to find the transformed form of the simple operations. The simple operations can be transformed back to their corresponding complex operations by modifying the *i*th index in the position vector of the original composite operation with the position parameter of the transformed form of the simple operation.

Therefore, the same transformation functions working for linear structures, such as the ones in (Suleiman et al. 1997; Oster et al. 2006), can be recursively applied in our treeOPT and asyncTreeOPT approaches.

5.3. Log compression

In operation-based merging, keeping the size of logs small is very important. A small log requires less time for the transfer of the log over the network and for the re-execution of the operations in the log. We apply a log compression procedure for reducing the size of the log by means of transforming several lower level operations into a single higher level operation. The compression procedure is called before an update or commit is performed. For instance, several *insertChar*

operations which insert characters in the same word can be grouped into one single *insertWord* operation inserting the word formed by the target characters of the *insertChar* operations. In the same way, we can combine several *insertWord* operations into a single *insertSentence* and several *insertSentence* operations into a single *insertParagraph*.

6. Applications of multi-level editing

In this section, we present the application of our multi-level editing approaches to providing awareness information and for handling conflicting changes that refer to certain parts of the document.

6.1. Support for awareness

Multi-level editing supports the computation of awareness information on different semantic levels of a text document. In (Papadopoulou et al. 2006), a novel approach for increasing awareness in collaborative authoring over hierarchical text documents based on the notion of editing profiles has been presented. These profiles provide an overview of the activity of users in different parts of the document. Metrics for computing the amount of changes on specific parts and levels of the document were introduced. The history distributed throughout the tree makes the computation of the amount of changes that refer to a certain unit of the document relatively simple. Operations referring to certain nodes can be easily scanned and filtered according to required criteria as they are kept in the history associated with the corresponding node. It would be more difficult to compute the metrics using a linear history buffer as each time an operation is integrated the whole history has to be scanned to determine the nodes whose metrics have to be changed. As awareness information might have to be computed in real-time, using a distributed history buffer instead of a linear one is a significant improvement in terms of efficiency.

6.2. Support for conflict handling

Multi-level editing also offers good support for handling conflicting changes referring to certain parts of the document. In (Ignat and Norrie 2006b), we describe our solution for handling conflicts in asynchronous communication by using the multi-level editing approach. Due to space constraints, we do not present the algorithmic details for handling conflicts here, but describe this aspect from an abstract point of view, such as how conflicts can be defined and resolved from the user interface. The user is asked to refer to (Ignat and Norrie 2006b) for details about how the *update* procedure described in section 5.1 was extended to deal with conflicts.

In the tree document model, conflicts can be defined at different granularity levels: paragraph, sentence, word or character. In our current implementation, we defined that two operations conflict if they modify the same semantic unit: paragraph,

sentence, word or character. The semantic unit is indicated by the granularity level chosen by the user. If a semantic unit is deleted, conflicts between the deleted unit and concurrent changes performed on the deleted unit can be defined only at a higher level. For instance, if a conflict should be detected between sentence deletion and word insertion in that sentence, the granularity level should be set to paragraph. If the granularity is set at the sentence level, the sentence is automatically deleted. Conflicts can be visualised at the chosen granularity level or at a higher level of granularity. For instance, if a user chooses to work at the sentence level it means that any two concurrent operations modifying the same sentence are in conflict. Conflicts can be presented at the sentence level so that the user can choose between the two versions of a sentence. It may happen that in order to choose the right version, the user has to read the whole paragraph to which the sentence belongs, i.e. the user can choose to visualise the conflicts also in the context of the paragraph or at a higher level. However, other rules for defining conflicts can be specified by the implementation of the *semanticConflict* function used by FORCE approach and mentioned in section 2.2.2. For instance, the *semanticConflict* function could check if some grammar rules are satisfied, a test that can be implemented easily using the semantic units defined by the hierarchical model.

We now describe the conflict resolution policies implemented by our system. The main distinction between conflict resolution policies is given by whether conflicts are resolved automatically or manually. Automatic resolution of conflicts means that users are not prompted for a decision regarding any kind of conflict and the result is obtained by automatically applying a merging policy. Manual resolution, on the other hand, means that, if conflicts among operations arise, users are asked to manually choose one version or the other.

If users choose automatic resolution, the default behaviour is that local operations are the ones to be kept in the case of conflict. Another policy for automatic resolution is to keep remote operations.

Concerning manual resolution policies, users can choose between operation comparison and conflict unit comparison policies. The operation comparison policy means that when two operations are in conflict, the user is presented with the effects of both operations and has to decide which of the effects to preserve. This is, however, not a user friendly approach for presenting conflicts. Consider the case of two users concurrently inserting three characters into the same word. This yields three *insertChar* operations on each site. Since the lowest conflict unit is the word, all operations from one site are in conflict with all operations on the other. The user is prompted to choose between all pairs of conflicting operations in order to decide on the final version of the word. This is not a quick method of solving conflicts. Moreover, it is also likely that if one user modified a word by adding a few characters to it and the other user modified it in a different way, the user performing merging probably wants either one word or the other, not a combination of them. This means that the user would choose either all local operations modifying the initial word or all remote operations modifying the

word and not a combination of the two sets of operations. Therefore, this technique is more a debugging conflict resolution policy than one for a real user.

In the conflict unit comparison policy, users have to choose between the set of all local operations and the set of all remote operations affecting the selected conflict unit (word, sentence or paragraph). The user is presented with the two different effects achieved by applying all the local operations and all the remote operations, respectively, pertaining to the conflict unit. The user can then choose between the two alternatives. By changing the conflict unit, the user can decide how many choices to make when performing an update: the higher up in hierarchy the conflict unit, the fewer the number of choices.

The rules for the definition of conflict and the policies for conflict resolution can be specified by each user before an update is performed and they do not have to be uniquely defined for all users. Moreover, for different update steps, users can specify different definition and resolution merge policies.

In what follows we illustrate the merging process in the case of asynchronous communication including conflict handling by means of an example. Assume the repository contains as version V_0 the document illustrated in Figure 5 and further that the document consists of ten paragraphs, each paragraph being formed by a set of sentences, each sentence consisting of a set of words and each word consisting of a set of characters.

Further suppose a conflict is defined between two operations concurrently modifying the same word and the policy of merging is that, in the case of conflict, local modifications are kept automatically. Further, assume two users check out version V_0 from the repository into their private workspaces. In order to explain in detail the functioning of the merging algorithm, we consider that the users are concurrently editing the last paragraph, paragraph 10 of the initial document “Our algorithm applie a linear merging procedure.” The first user performs the operations O_{11} and O_{12} , where $O_{11} = \text{insertChar}(\text{“d”}, 10, 1, 3, 7)$ and $O_{12} = \text{insertWord}(\text{“recursively”}, 10, 1, 4)$. Operation O_{11} inserts the character “d” in the tenth paragraph, first sentence, third word, as the seventh character and operation O_{12} inserts the word “recursively” into tenth paragraph, first sentence, as the fourth word in order to obtain the version

[...]

Our algorithm applied *recursively* a linear merging procedure.

where [...] denotes the other document paragraphs that have not been modified. The second user performs the operations $O_{21} = \text{insertSentence}$

Version control systems are widely used to support a group of people working together on a set of documents over a network.

[...]

Our algorithm applie a linear merging procedure.

Figure 5. Example document.

(“*The approach offers an increased efficiency.*”,10,2) and $O_{22} = \text{InsertChar}$ (“s”,10,1,3,7) in order to obtain

[...]

Our algorithm applies a linear merging procedure. *The approach offers an increased efficiency.*

Suppose that both users try to commit, but $User_1$ gets access to the repository first, while $User_2$'s request is queued. After the commit operation of $User_1$, the last version in the repository becomes $V_1 = \text{“[...]Our algorithm applied recursively a linear merging procedure.”}$. DL_{10} representing the difference between V_1 and V_0 in the repository is obtained as a result of the linearisation of the history buffer distributed throughout the tree, $DL_{10} = [O_{12}, O_{11}]$.

When $User_2$'s request is processed, the repository sends to $User_2$ a message to update the local copy. In order to update the local copy of the document, the *update* procedure is applied. The local tree generated at the site of $User_2$ is traversed in a top-down manner. First the document level history is analysed. There are no remote operations of insertions or deletions of paragraphs to be merged. The update is then applied to the paragraph level. As all operations to be merged refer to paragraph 10, the log of paragraph 10 is analysed. Again, there are no remote operations of sentence level, so the processing is applied to the sentence level. The local document contains two sentences, but there are no remote operations referring to sentence 2, so the merging for sentence 1 is further analysed. Operation O_{12} is of word level, and because there are no local operations of word level, O_{12} keeps its original form and is executed locally. The update procedure is recursively applied for each of the words belonging to sentence 1. We analyse only the update applied to the third word of sentence 1, since the remote logs corresponding to the other words in the sentence are empty. The merge procedure is applied between the list of operations consisting of O_{11} and the list consisting of O_{22} . O_{11} and O_{22} are conflicting and, according to the assumed policy, the local operation is kept. As a result of this merging, for the third word of sentence 1, the new local log NLL , i.e. the list of operations to be transmitted to the repository, is $[inv(O_{11}), O_{22}]$ and the new remote log NRL , i.e. the list of operations to be applied to the local copy of the document will be empty. Therefore, the new local version of the document in the workspace of $User_2$ becomes “[...]Our algorithm applies recursively a linear merging procedure. The approach offers an increased efficiency.” This version of the document represents also the new version V_2 of the document in the repository after $User_2$ commits. DL_{21} , the difference between V_2 and V_1 , is $DL_{21} = [O_{21}, inv(O_{11}), O_{22}]$.

When $User_1$ updates his local version of the document, the operations belonging to D_{21} are executed locally.

This example illustrates the fact that only a small number of transformations have to be performed using a tree-model of the text document with the local log distributed throughout the tree. The operations of a specific granularity do not need to be transformed against the operations of lower level granularity. The

performance gain obtained by using a tree representation compared to using the linear representation of text documents increases with the number of operations to be merged. In this example we have also seen that it is easy to define generic conflict rules involving different semantic units, such as specifying that concurrent insertions in the same word are conflicting.

Note that systems such as CVS and Subversion that use the diff tool for merging will detect conflict between the two versions of the document. Changes from the two versions of the document refer to the same paragraph, i.e. to the same line, and the conflict unit defined in these systems is the line. The user has then to choose one of the two versions. In the case that a combined effect of the changes that have been performed is desired, the user has to manually add the changes performed on the version that was not selected.

Other conflict definition and resolution rules can be defined. For instance, if conflict is defined at the level of word as in the previous example, but the manual resolution policy is chosen, when $User_2$ updates the local workspace, they are presented with the two versions of the initial word “applie”, i.e. the local version “applies” and the version in the repository “applied”. If the user chooses the local version of the word, the final version of the document will be “[...]Our algorithm applies recursively a linear merging procedure. The approach offers an increased efficiency.”

If conflict is defined at the level of the sentence and also the manual policy for merging is chosen, when $User_2$ updates the local workspace, they are presented with the two versions of the initial sentence “Our algorithm applie a linear merging procedure”, i.e. the local version “Our algorithm applies a linear merging procedure” and the remote version “Our algorithm applied recursively a linear merging procedure”. If the local version is chosen, the final version of the document will be “[...]Our algorithm applies a linear merging procedure. The approach offers an increased efficiency.”

7. Complexity analysis

In this section, we analyse the complexity of our multi-level editing algorithms with respect to the complexity of the algorithms that use a linear history buffer, for both real-time and asynchronous communication.

7.1. treeOPT versus merging algorithms for linear structures

In what follows we analyse the complexity of GOT and SOCT2 as representative algorithms for linear structures for real-time communication and then the complexity of the treeOPT algorithm. The complexity of treeOPT is first computed with respect to the complexity of any merging algorithm for linear structures and then the general complexity is instantiated with the complexity of GOT and SOCT2.

7.1.1. GOT

As mentioned in section 2.2.1, the GOT algorithm (Sun et al. 1998) is divided into three parts. The first part undoes the operations that strictly follow the remote operation O_{new} , the second one deals with the execution of O_{new} and the third part with the redoing of the undone operations. In the second and third part of the GOT algorithm, the transformation functions *LIT* and *LET* of including, and respectively excluding, a list of operations from another list of operations are repeatedly used. The complexity of these transformation functions is $O(p \cdot q)$, where p and q are the sizes of the two lists of operations.

In the undo/do/redo scheme of the GOT algorithm, operations from the history buffer $HB = [EO_1, \dots, EO_m, \dots, EO_n]$ are undone from right to left till an operation EO_m totally preceding the remote operation O_{new} is encountered. This step is of complexity $n - m$. Afterwards the execution form of the remote operation is computed.

We analyse next the complexity of the second part of the GOT algorithm which consists of transforming operation O_{new} against the list $HB' = [EO_1, EO_2, \dots, EO_m]$ of operations which precede O_{new} in the total order. O_{new} has to be transformed against concurrent operations in HB' that have a different context than O_{new} . The list HB' is scanned from left to right to find the first operation EO_k that is concurrent with O_{new} and afterwards to find the list $EOL = [EO_{c1}, EO_{c2}, \dots, EO_{cr}]$ of operations that follow EO_k but that precede O_{new} . This step requires m comparisons. Operations in EOL have to be excluded from the context of O_{new} . In order to satisfy the precondition of the exclusion transformation, the operations from EOL have to be transformed to first exclude the effect of the concurrent operations with O_{new} . Let us denote the result of this transformation by $EOL' = [EO'_{c1}, EO'_{c2}, \dots, EO'_{cr}]$. Each element EO'_{ci} is computed by first excluding from EO_{ci} the effect of the list of operations from HB' situated at its left side starting with index k and then including $[EO'_{c1}, EO'_{c2}, \dots, EO'_{ci-1}]$. This step requires $\sum_{i=1}^r ((c_i - k) + i - 1)$ transformations. For the worst case, where $r = m - k$, the complexity of this step is $(m - k)^2$. Afterwards, the list EOL' has to be excluded from O_{new} , which requires r transformations and operation O_{new} has to include the operations from HB' starting with index k , which requires $m - k$ transformations. Therefore, the complexity for the computation of the execution part of the remote operation in the GOT algorithm is $O((m - k)^2)$.

We analyse next the complexity for redoing the undone operations. When operations are redone, they have to exclude the effect of the previous operations in the history buffer that were undone and then include the effect of the remote operation and of the previous operations that have been redone. This step involves $\sum_{i=1}^{n-m} ((i - 1) + i)$ transformations and has a complexity of $O((n - m)^2)$.

Consequently, the whole algorithm for the integration of a remote operation into the history buffer has a complexity of $O(n^2)$, where n is the length of the history buffer.

7.1.2. SOCT2

We now analyse the complexity of the SOCT2 algorithm. As mentioned in section 2.2.1, the SOCT2 algorithm for the integration of a remote operation into the history buffer consists of two steps. In the first step of the algorithm, the operations in the history buffer are reordered so that the operations that precede the remote operation are situated before the operations concurrent with the remote operations. The reordering procedure considers each operation in the history buffer and, if that operation precedes the remote operation, it is transposed towards the beginning of the history at the end of the sequence of operations that precede the remote operation. The second step of the algorithm consists of transforming the remote operation against the concurrent operations in the history buffer. The worst case scenario in the reordering process when the most number of transpositions have to be performed is the one when the operations that precede the remote operation are situated at the end of the history. Suppose m operations precede the remote operation. Each of these operations has to be transposed against $n - m$ operations. In total there will be $m(n - m)$ transpositions to be performed. Therefore, the process of reordering is of complexity $O(n^2)$, where n is the size of the history buffer and the maximum number of transformations is obtained if $m = n/2$. The second step of the SOCT2 algorithm of integrating the remote operation into the history buffer consists of the forward transposition of the remote operation according to the sequence of concurrent operations and is of order $O(n)$. Therefore, the complexity of the SOCT2 algorithm, i.e. of the integration of a remote operation into the history buffer, is $O(n^2)$, where n is the length of the history buffer.

7.1.3. treeOPT

We can now compare the complexity of the treeOPT algorithm with respect to the complexity of the operational transformation algorithms working on a linear structure, such as GOT and SOCT2 whose complexities we previously analysed.

The complexity of the treeOPT algorithm in the worst case equals the complexity of the SOCT2 and GOT algorithms. The worst case occurs when the whole history is concentrated on a single element in the tree and the granularity of that element is the lowest existing granularity. This worst case corresponds to the case when all users involved in the collaboration concurrently edit the same word in the document, by adding or removing characters belonging to that word. In this case, the whole history of the document is the history attached to the word node and it has to be traversed each time a transformation has to be performed. Therefore, the complexity is $O(n^2)$, where n is the number of operations performed, and the complexity is the same as in the case of linear transformation approaches. But, the worst case rarely occurs in practice.

We therefore analyse the complexity of the treeOPT algorithm in the general case where we consider that the tree document contains p operations of document level, i.e. insertions and deletions of paragraphs, s operations of paragraph level,

i.e. insertions and deletions of sentences, w operations of sentence level, i.e. insertions and deletions of words and c operations of word level, i.e. insertions and deletions of characters. Further, suppose that we have the following average counters concerning the structure of the document. Suppose that pc is the average number of paragraphs in a document, sc is the average number of sentences in a paragraph, wc the average number of words in a sentence and cc the average number of characters in a word. If the remote operation is an operation of document level, only the document history has to be traversed and transformations performed only at document level. A linear transformation algorithm is applied on the document history. Assume that the complexity of the linear transformation algorithm is $C(n)$ where n is the number of operations in the history buffer. Therefore, the complexity of the integration of a remote operation referring to a paragraph is $C(p)$. If the remote operation is of paragraph level, a linear operational transformation approach has to be applied for the document history and afterwards for the paragraph history where the operation belongs. Since there are s operations referring to sentences and pc is the average number of paragraphs in a document, the number of operations associated with a paragraph is s/pc . Therefore, the complexity of the integration of a remote operation of paragraph level is $C(p) + C(s/pc)$. Similarly, the complexity of the integration of a remote operation of sentence level is $C(p) + C(s/pc) + C(w/(pc \cdot sc))$. And the complexity for the integration of a remote operation of word level is $C(p) + C(s/pc) + C(w/(pc \cdot sc)) + C(c/(pc \cdot sc \cdot wc))$.

The complexity for the integration of a remote operation in the case of a linear algorithm that could deal with operations of different levels of granularity such as document, paragraph, sentence and word is $C(p + s + w + c)$. The majority of existing linear operational transformation algorithms deal only with operations that target characters and therefore operations targeting words, sentences or paragraphs have to be split into operations targeting characters. For instance, an operation of insertion or deletion of a word would be divided into cc operations targeting the characters of the word. Therefore, the complexity of such an algorithm is $C(p \cdot sc \cdot wc \cdot cc + s \cdot wc \cdot cc + w \cdot cc + c)$. Let us consider the case of the SOCT2 algorithm. As shown in section 7.1.2, the complexity of SOCT2 is $C(n) = n^2$, where n is the size of the history buffer. As presented in (Suleiman et al. 1997), transformation functions for operations targeting characters have been proposed and therefore the complexity for the integration of a remote operation into a history representing the insertions or deletions of p paragraphs, s sentences, w words and c characters is $(p \cdot sc \cdot wc \cdot cc + s \cdot wc \cdot cc + w \cdot cc + c)^2$. If SOCT2 is used as the linear algorithm recursively applied by treeOPT, the complexity for the integration of a remote operation referring to a character is $p^2 + (s/pc)^2 + (w/(pc \cdot sc))^2 + (c/(pc \cdot sc \cdot wc))^2$. We see that treeOPT has a much better complexity than the existing algorithms using a linear history buffer.

However, in real-time communication, as pointed out in (Sun et al. 1998), the history buffer can be kept to a small size as a garbage collection scheme can be

applied to remove those operations in the history buffer which are no longer concurrent with any future operations. In this case, the improvements of treeOPT with respect to algorithms that deal with hierarchical document structures, but use a linear history buffer, might not be very significant. Concerning efficiency, the advantage of multi-level editing over approaches using a linear history buffer can be especially seen in asynchronous communication where a lot of concurrent operations can be generated when users work in isolation, as shown in the next section.

7.2. asyncTreeOPT versus merging algorithms for linear structures

In this section, we analyse the complexity of linear operational transformation algorithms such as SOCT2 and GOT which are usually used for real-time communication in the event that they are adapted for asynchronous communication with a central repository, as well as the complexity of the FORCE algorithm specialised for asynchronous communication. We then compute the complexity of our asyncTreeOPT algorithm with respect to any linear algorithm for merging. To illustrate the efficiency of asyncTreeOPT versus FORCE, the complexities of the two algorithms are instantiated for certain document structures and numbers of concurrent operations.

7.2.1. Algorithms for merging linear structures

Algorithms adapted for asynchronous communication should deal with conflicts. As shown in section 2.2.2, in order to update the local version of the document, the local log LL containing the operations executed in the local workspace has to be merged with the remote log DL containing the operations concurrently performed with the local operations and committed in the repository. Operations in DL have to be transformed against operations in LL before they can be locally applied, and operations in LL have to be transformed against operations in DL in order to compute the new delta in the repository. Suppose that a conflict between an operation O_{di} in the remote log DL and an operation in the local log LL exists and the local operation has to be cancelled. The cancelled local operation should be excluded from the operations that follow it in LL so that when these operations are transformed against operations in DL they reflect the fact that an operation preceding them was cancelled. Therefore, a mechanism for performing the integration of an operation into the log and the cancellation of an operation from the log has to be provided by an algorithm adapted for asynchronous communication.

In (Prakash and Knister 1994; Sun 2002), mechanisms for performing undo have been proposed, so one of these mechanisms could be used for the cancellation of an operation. For the integration of an operation into a log, one of the algorithms working for real-time communication such as SOCT2 (Suleiman et al. 1997) or GOTO (Sun and Ellis 1998) could be applied. However, specialised algorithms for asynchronous communication such as FORCE (Shen and Sun 2002) achieve a better performance as shown below.

Let us reconsider the example illustrated in Figure 3. We denote that the remote log DL and the local log LL have the following structure: $DL = [O_{d1}, \dots, O_{di}, \dots, O_{dm}]$ and $LL = [O_{l1}, \dots, O_{li}, \dots, O_{ln}]$. The operations in DL and LL respectively were sequentially executed and therefore they are contextually preceding. As shown in Figure 3, we supposed that operations in DL and LL were generated on the same version of the document and therefore O_{d1} and O_{l1} have the same initial context. Moreover, all operations in DL are concurrent with the operations in LL . Let us analyse the number of transformations that have to be performed to integrate each operation belonging to DL into LL and each operation belonging to LL into DL using the SOCT2 (Suleiman et al. 1997) algorithm. Let us analyse first the integration of the operations belonging to DL into LL . When O_{d1} is transformed against all operations in LL , n inclusion transformations will be performed, the result being operation O'_{d1} . When O_{d2} has to be integrated into the transformed local log $LL' = [O_{l1}, \dots, O_{ln}, O'_{d1}]$, the operations in the log have to be reordered so that the first part of the log contains the operations that precede O_{d2} and the last part of the log contains the operations that are concurrent with O_{d2} . Therefore, O'_{d1} has to be transposed at the beginning of the history buffer. Each step of the transposition involves the computation of an inclusion and exclusion transformation and, therefore, the transposition process requires $2 \cdot n$ transformations. Afterwards, O_{d2} has to be transformed against the concurrent operations and, in this case, n inclusion transformations will be performed. Therefore, the integration of O_{d2} requires $3 \cdot n$ transformations and the integration of all operations in DL into LL requires $n + 3 \cdot n \cdot (m - 1) = 3 \cdot n \cdot m - 2 \cdot n$ transformations to be performed. Similarly, the integration of all operations belonging to LL into DL requires $3 \cdot n \cdot m - 2 \cdot m$ operations to be performed. Therefore, the total number of transformations are $6 \cdot n \cdot m - 2 \cdot m - 2 \cdot n$.

The FORCE (Shen and Sun 2002) approach transforms each operation O_{di} in DL in turn with respect to each operation O_{lj} in LL and, after such a transformation is performed, the symmetric transformation of O_{lj} with respect to O_{di} is also performed. The approach requires $2 \cdot n \cdot m$ transformations to be performed and the logs have to be traversed only once.

7.2.2. *asyncTreeOPT*

We now analyse the complexity of the *update* procedure of the *asyncTreeOPT* algorithm. Suppose that the local log contains p_1 operations of document level, i.e. insertions and deletions of paragraphs, s_1 operations at paragraph level, i.e. insertions and deletions of sentences, w_1 operations of sentence level, i.e. insertions and deletions of words and c_1 operations of word level, i.e. insertions and deletions of characters. Further suppose that the remote log contains p_2 operations of document level, s_2 operations of paragraph level, w_2 operations of sentence level and c_2 operations of word level. Suppose that the document structure is characterised by the following average counters which remain constant as a result of the modifications in the tree: pc is the average number of paragraphs in a document, sc the average number of sentences in a paragraph, wc the average number of words in a sentence

and cc the average number of characters in a word. We denote the complexity of the merge procedure between two lists of operations containing n and m operations, respectively, by $C(n, m)$. When the update procedure is applied for the document level, the local list of operations referring to paragraphs is merged with the remote list of operations referring to paragraphs and therefore the complexity of $C(p_1, p_2)$ is obtained. Further, the operations in the remote log following the operations that refer to paragraphs have to be transformed against the operations in the local log targeting paragraphs. For this step, $p_1 \cdot (s_2 + w_2 + c_2)$ transformations are performed. The transformed operations in the remote log referring to sentences, words and characters are divided among the paragraphs in the document. By supposing that operations are equally distributed throughout the document structure, the number of operations referring to sentences, words or characters in the new remote log that target a certain paragraph are $\frac{s_2 + w_2 + c_2}{pc}$. The s_1 operations in the local log of paragraph level, i.e. insertions and deletions of sentences, are equally divided between the pc paragraphs and therefore the history associated with a certain paragraph contains $\frac{s_1}{pc}$ operations. Next, the update procedure is applied for the level of paragraph. For each paragraph, the merge between the local list of operations of paragraph level and the remote list of operations of paragraph level is performed, the total number of transformations being $C(\frac{s_1}{pc}, \frac{s_2}{pc})$. Afterwards the operations in the remote log of sentence and word level are transformed against the operations in the local log of paragraph level and therefore $\frac{s_1}{pc} \cdot \frac{w_2 + c_2}{pc}$ are performed. At the level of paragraph, the total number of operations performed is $pc \cdot (C(\frac{s_1}{pc}, \frac{s_2}{pc}) + \frac{s_1}{pc} \cdot \frac{w_2 + c_2}{pc})$. The update procedure is then recursively applied for the sentence and word level. The total number of transformations is

$$C(p_1, p_2) + p_1 \cdot (s_2 + w_2 + c_2) + pc \cdot \left(C\left(\frac{s_1}{pc}, \frac{s_2}{pc}\right) + \frac{s_1}{pc} \cdot \frac{w_2 + c_2}{pc} + sc \cdot \left(C\left(\frac{w_1}{pc \cdot sc}, \frac{w_2}{pc \cdot sc}\right) + \frac{w_1}{pc \cdot sc} \cdot \frac{c_2}{pc \cdot sc} + w_c \cdot C\left(\frac{c_1}{pc \cdot sc \cdot wc}, \frac{c_2}{pc \cdot sc \cdot wc}\right) \right) \right)$$

The complexity of the FORCE algorithm for merging two lists of operations of size n and respectively m is $C(n, m) = 2 \cdot n \cdot m$ as shown in section 7.2.1. If the FORCE algorithm for merging is used by the asyncTreeOPT algorithm recursively over the document structure, the number of transformations that would be performed is

$$\begin{aligned} & 2 \cdot p_1 \cdot p_2 + p_1 \cdot (s_2 + w_2 + c_2) + pc \cdot \left(2 \cdot \frac{s_1}{pc} \cdot \frac{s_2}{pc} + \frac{s_1}{pc} \cdot \frac{w_2 + c_2}{pc} + sc \cdot \left(2 \cdot \frac{w_1}{pc \cdot sc} \cdot \frac{w_2}{pc \cdot sc} + \frac{w_1}{pc \cdot sc} \cdot \frac{c_2}{pc \cdot sc} + w_c \cdot \left(2 \cdot \frac{c_1}{pc \cdot sc \cdot wc} \cdot \frac{c_2}{pc \cdot sc \cdot wc} \right) \right) \right) \\ & = 2 \cdot p_1 \cdot p_2 + p_1(s_2 + w_2 + c_2) + 2 \cdot \frac{s_1 \cdot s_2}{pc} + \frac{s_1(w_2 + c_2)}{pc} \\ & \quad + 2 \cdot \frac{w_1 \cdot w_2}{pc \cdot sc} + \frac{w_1 \cdot c_2}{pc \cdot sc} + 2 \cdot \frac{c_1 \cdot c_2}{pc \cdot sc \cdot wc} \end{aligned}$$

Consider the case that the local log contains 50 operations distributed as follows: five operations of insertions/deletions of paragraphs, 15 operations of insertions/deletions of sentences, 20 operations of deletions/insertions of words and 10 operations of insertions/deletions of characters. Suppose the remote log contains 100 operations distributed as follows: 15 operations of insertions/deletions of paragraphs, 30 operations of insertions/deletions of sentences, 35 operations of insertions/deletions of words and 20 operations of insertions/deletions of characters. Consider that the document has the following characteristics: a paragraph contains an average number of $sc = 5$ sentences, a sentence contains an average number of $wc = 20$ words and a word contains an average number of $cc = 5$ characters. These average values are usually constant and what mainly distinguishes documents is the total number of paragraphs in a document. Therefore, we considered the average number of paragraphs in the document pc variable and in Table 1 we provide the total number of transformations that have to be performed for different values of parameter pc .

The FORCE approach proposed in (Shen and Sun 2002) was described for operations defined on strings. The operations in our previous example, targeting paragraphs, sentences, words and characters will be translated into operations on strings in the FORCE approach. For instance, an operation of insertion of a sentence will be transformed into an operation of insertion of the string representing the content of the sentence. In this case, the number of transformations performed in the FORCE approach for our example is $2 \cdot 50 \cdot 100 = 10\,000$. For the case that operations would be defined only for characters, as is the case for most operational transformation approaches, the total number of transformations would be $2 \cdot (5 \cdot 5 \cdot 20 \cdot 5 + 15 \cdot 20 \cdot 5 + 20 \cdot 5 + 10) \cdot (15 \cdot 5 \cdot 20 \cdot 5 + 30 \cdot 20 \cdot 5 + 35 \cdot 5 + 20) = 87\,912\,900$. Table 1 shows the total number of transformations in the `asyncTreeOPT` approach for different values of the number of paragraphs pc in the document. The table also shows the number of transformations

Table 1. Estimated number of transformations in `asyncTreeOPT` and FORCE approach when operations are defined on strings and on characters.

No. of paragraphs (pc)	No. of transf. in <code>asyncTreeOPT</code>	No. of transf. in FORCE (op. on strings)	No. of transf. in FORCE (op. on characters)
1	2,664	10,000	87,912,900
2	1,619	10,000	87,912,900
4	1,097	10,000	87,912,900
5	992	10,000	87,912,900
10	783	10,000	87,912,900
20	679	10,000	87,912,900
50	616	10,000	87,912,900
100	595	10,000	87,912,900

required in these cases for the FORCE approach when operations are defined on strings and on characters. We see that the number of transformations performed using the `asyncTreeOPT` approach is significantly lower than the number of transformations performed using the FORCE approach.

Providing performance measurements is out of the scope of this paper. In (Li and Li 2006, 2007), the authors provide quantitative studies that present which factors contribute to algorithm performance and show that algorithm performance is directly proportional to its complexity in terms of the number of transformations computed.

8. Open issue – the split/join problem

In this section, we report on some problems that we encountered when adapting the `treeOPT` and `asyncTreeOPT` algorithms for hierarchical text documents and the solutions we adopted to overcome these problems.

Even though the algorithms work very well with insert and delete primitives at different levels of the hierarchy, in practice, these two primitives are not sufficient to perform all possible operations. Let us consider the following example. Assume the second paragraph of a document consists of the following sentence: “Merging is flexible and efficiency is obtained.” shown in the left part of Figure 6.

Suppose a user splits the sentence “Merging is flexible and efficiency is obtained.” into two sentences: “Merging is flexible.” and “and efficiency is obtained.” One alternative of simulating the split operation is to first delete the words “and”, “efficiency”, “is” and “obtained”, from the first sentence, and then to insert the whole sentence: “and efficiency is obtained.”. As a result of performing these operations, the new structure of paragraph 2 is illustrated in the right part of Figure 6.

Unfortunately, this approach does not work directly as desired. Suppose that concurrently with the split operation of the sentence, another user inserts the word “better” as the fifth word in the sentence in order to obtain “Merging is flexible

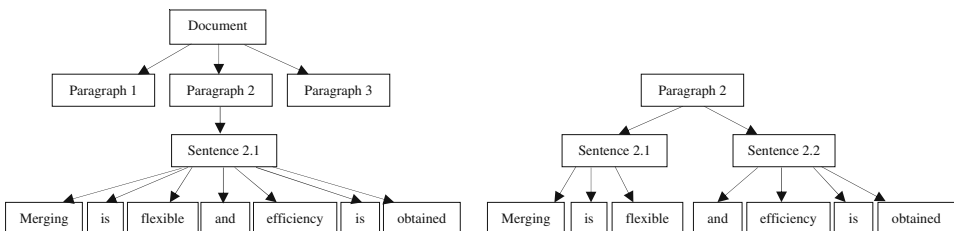


Figure 6. Example of split problem.

and better efficiency is obtained.” The operation sequence is illustrated in the left part of Figure 7.

As we can see, by the time operation *insertWord(2,1,5,“better”)* is received at *Site₁*, the words “and”, “efficiency”, “is” and “obtained” are already deleted from the paragraph 2, sentence 1, and these operations of word deletion are kept in the history of sentence 2.1. By applying the treeOPT algorithm, the operation *insertWord(2,1,5,“better”)* is transformed into *insertWord(2,1,4,“better”)*. The resulting structure of the paragraph, shown in the right part of Figure 7, is not what the user at *Site₂* intended.

A solution to the split problem would be to delete the unit that has to be split and insert two new units containing the split parts. For instance, in the previously described example of splitting the sentence “Merging is flexible and efficiency is obtained.”, the solution would be to first delete the sentence by generating the operation $O_1 = \text{deleteSentence}(2,1)$ and then insert two new sentences “Merging is flexible.” and “and efficiency is obtained.”, by issuing the operations $O_2 = \text{insertSentence}(2,1, \text{“Merging is flexible.”})$ and $O_3 = \text{insertSentence}(2,2, \text{“and efficiency is obtained.”})$. The problem is that executing these three operations has to be atomic at all sites, which is very hard to ensure in a real-time system, where operations arrive with different delays. For instance, if a user at one site sees only the effect of O_1 and O_2 , the effect of O_3 being delayed, the user could misinterpret the action of splitting the sentence. By deleting the original sentence, all concurrent operations referring to the original sentence are cancelled. Moreover, transformation functions would have to be adapted for groups of operations.

Some other possible ways of simulating the split operation using only insertions and deletions exist, but none of them are feasible. The reason is that a structural element might appear to be different on two hosts at the same time, and the two structures converge only because the history of operations on that element is kept at both sites. When an element is split into two parts, its history must also be split. By using only elementary insert/delete operations, the cases when the history needs to be split or not cannot be detected. Operations of insert and delete do not determine the splitting of a history buffer. Only the introduction of a new split primitive would trigger the history buffer to be split. The same

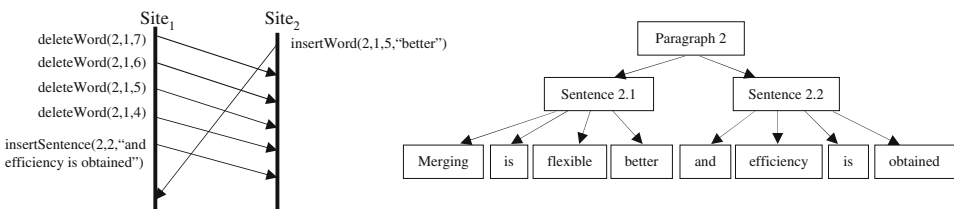


Figure 7. Erroneous result due to split operation.

problem is encountered in the case of joining two elements. For example, if we delete a sentence separator, the two adjacent sentences will be joined into a single one implying the joining of the histories of the two sentences.

An alternative solution would be to introduce two other primitives: split and join, and to modify the algorithm by implementing operational transformation functions for these primitives as well. By means of an example, we show that this solution is not feasible either. Suppose that the initial state of the document, as in the previous example, consists of the sentence S_1 ="Merging is flexible and efficiency is obtained." and that both local copies of document at $Site_1$ and $Site_2$ contain sentence S_1 . A scenario illustrating the concurrent editing of the document is given in Figure 8.

Operation O_1 initiated by the user at $Site_1$ splits sentence S_1 into two sentences S_{11} ="Merging is flexible" and S_{12} ="and efficiency is obtained.". Operation O_2 deletes sentence S_{12} . When operation O_2 arrives at $Site_2$, it has to be transformed against O_3 , whatever O_3 is. But the precondition of the transformation is that O_2 has the same initial context as O_3 . In the case that the GOT algorithm is applied, the intuitive explanation of the action that has to be performed is that O_1 has to be excluded from the context of O_2 . But, let us analyse in detail how the GOT algorithm is applied at $Site_2$. Suppose that O_3 totally precedes O_1 which totally precedes O_2 . When O_1 arrives at $Site_2$, it is transformed against O_3 , the result of the transformation being O'_1 . The history buffer is $HB = [O_3, O'_1]$. When O_2 arrives at $Site_2$, both O_3 and O'_1 are preceding in the total order O_2 , so no operations have to be undone. O_3 is concurrent with O_2 and O'_1 is preceding O_2 . According to the GOT algorithm, O'_1 is transformed to exclude operation O_3 from its context, resulting in operation O''_1 which should return the original operation O_1 . Operation O_2 should then exclude the effect of O''_1 in order to include afterwards the effect of $[O_3, O'_1]$. The problem that occurs is when O_2 excludes the effect of $O''_1 = O_1$. If we exclude the effect of O_1 that splits S_1 into S_{11} and S_{12} , the deletion of the sentence "and efficiency is obtained." will have to be transformed into four different operations which are not of the same granularity level: *deleteWord*("and"), *deleteWord*("efficiency"), *deleteWord*("is")

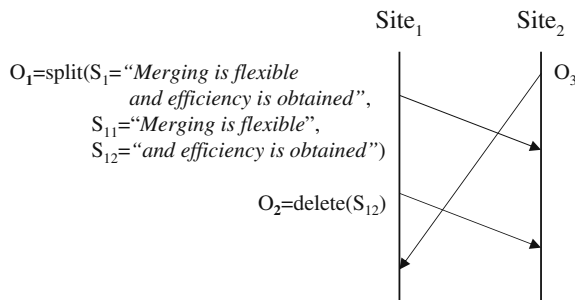


Figure 8. Counterexample for split primitive.

and *deleteWord*("obtained"). For simplicity, we did not specify the positions of word deletions. This is not an acceptable solution as the idea of the treeOPT algorithm is to transform operations against other operations at the same level of granularity, the result being an operation also of that level of granularity.

The previously described scenario where O_1 and O_2 are causally preceding operations (O_1 precedes O_2) generated by the same site and O_1 has to be excluded from O_2 occurs only in the GOT algorithm. However, this case does not appear in the SOCT2 family of algorithms since the order of execution between two causally preceding operations generated from the same site is never changed in these approaches.

Anyway, in all existing operational transformation approaches the case that a delete operation targeting a unit is transformed against a concurrent split operation applied on the same unit occurs, and may result in a composite operation consisting of two delete operations. Consider again the document consisting of the sentence S_1 ="Merging is flexible and efficiency is obtained.". Consider that $User_1$ performs the operation O_1 of splitting sentence S_1 into two sentences S_{11} ="Merging is flexible" and S_{12} ="and efficiency is obtained.". Consider that $User_2$ concurrently with the operation performed by $User_1$ issues operation O_2 to delete sentence S_1 . When operation O_2 is transformed against operation O_1 , the result is the composed operation containing the deletion of the sentence S_{11} and the deletion of the sentence S_{12} . Working with composite operations leads to the issue of defining the inclusion and exclusion transformation functions for groups of operations which still remains an open issue. Therefore, introducing the additional operations split and join is not a suitable solution.

The most appropriate solution we have so far found for real-time collaboration, although somehow disappointing, is not to split or join elements in the tree structure. For example, given the text "multi version" composed of two words, deleting the space between the two words still has as a result the two words, even though not separated by any separator. The same approach can be adopted in the case of splits. If we insert a sentence separator, for instance a dot, or even a paragraph separator such as new line inside a sentence, the text is kept as a single sentence. Embracing this approach leads however to degenerated elements. For example, the text "The approach offers flexibility. A better efficiency is obtained" might be a single large degenerated word, and the text "merge" can be stored in three degenerated sentences: "m", "er", and "ge". Obviously, the hierarchical structure resulting in the case of degenerated elements is different from the one obtained by parsing the text and delimiting the elements using their natural separators.

Even with this drawback, the algorithm works well. When issuing an operation, the positions of the elements of different granularity levels (paragraph, sentence and word) are computed by taking into account the length of the previous elements. Consequently, the fact that the elements are degenerated does not matter. The efficiency of the algorithm remains unaffected by the degenerated

elements, because the structure of the document remains hierarchical, and operations are transformed locally, spanning only a small part of the whole history of operations. Unfortunately, semantic consistency is more difficult to maintain. However, the problem is not as severe as it seems, as the reparsing of the whole document is performed every time a new user begins editing the same document, or when the document is reloaded. Reparsing restores the semantic consistency of elements, only non-degenerated elements being generated. Reparsing the document should be enforced as often as possible. But parsing the document implies having the same copy of the document at all sites. This means that reparsing can be performed only in moments of quiescence. Either the system can detect the moments of quiescence and initiate reparsing on copies of the document at all sites, or quiescence could be enforced by the system from time to time.

To avoid the join/split problem, the interface could restrict the user from performing these operations. We found that this solution is too frustrating for the user and therefore we adopted element degeneration as a solution.

The split/merge also remains an issue in asynchronous collaborative editing for our `asyncTreeOPT` algorithm. The solution that we have adopted in this case was to treat the split as a deletion of the old semantic unit and the insertion of the two new semantic units and the merge as the deletion of the two old semantic units and the insertion of the new semantic unit. However, the issue is not as critical as in the case of real-time editing, since there is not the possibility of interfering operations -merging involves the reciprocal transformation of two constant lists of operations.

9. Related work

As we already presented, there exist various approaches for maintaining consistency in real-time collaborative text editing that work with a linear structure of documents (Ellis and Gibbs 1989; Ressel et al. 1996; Sun et al. 1998; Suleiman et al. 1997; Sun and Ellis 1998; Vidot et al. 2000; Li and Li 2004, 2005).

A real-time operational transformation approach that has been extended for documents conforming to a hierarchical structure such as SGML (Standard General Markup Language) dialects including XML and HTML has been proposed in (Davis et al. 2002). In the proposed hierarchical model, a minimum number of nodes have an associated name, the other nodes being addressed by the path from their nearest named ancestor. The operations that can be performed are the insertion of a subtree as a child of a specified node, the deletion of a subtree and the modification of the contents of a node. Transformation functions have been defined for each pair of operations. The transformation functions are quite complex since operations can refer to any node in the tree document and different results have to be generated according to the relationships between paths of target nodes of operations. In our approach, the simple transformation

functions for linear structures, such as the functions proposed in (Suleiman et al. 1997), are used for maintaining consistency over hierarchical structures by their recursive application over the document levels.

Another operational transformation approach for the real-time collaborative editing of hierarchical documents such as XML has been implemented in the SAMS (Synchronous, Asynchronous and Multi-Synchronous System) environment (Molli et al. 2002). Transformation functions are also adapted for hierarchical structures.

An operational transformation approach for collaborative word processing has been proposed in (Sun et al. 2004; Sun et al. 2006), as part of the CoWord/CoPPT project. The word processing requires that the editing is not only performed on text, but also on graphics and images. The proposed approach assumes that the document conforms to a hierarchical structure (Xia et al. 2005; Sun et al. 2006).

As opposed to the approaches proposed in (Davis et al. 2002; Molli et al. 2002; Sun et al. 2004) and (Sun et al. 2006), we have used a linear operational transformation mechanism that has been recursively applied over the document levels. Keeping the history buffer distributed throughout the tree offers support for tracking user activity referring to certain nodes in the tree and for an increased efficiency.

Another approach for real-time collaboration that analysed the tree representation of documents as the basic unit for collaboration is the dARB approach (Ionescu and Marsic 2003). The approach uses a distributed arbitration mechanism for dealing with the consistency of the replicated copies of the document. The arbitration scheme cannot resolve all concurrent accesses to documents automatically and, in some cases, must resort to asking users to manually resolve inconsistencies. Rather than obtaining a partial combination of the intentions of the users that issued concurrent operations, the dARB approach preserves the intentions of only one user. In our approach, we preserve the intentions of all users even if the result might be a strange combination of all intentions. Despite the hierarchical structure of the document in the dARB approach divided into paragraphs, sentences, words and characters, the communication is done at the character level, i.e. an operation is generated each time a character is inserted. Our approach is not character-wise, but element-wise, i.e. the operations of insertion and deletion are performed at different granularity levels. In the arbitration mechanism of the dARB approach, there are cases when one site wins the arbitration and it needs to send not only the state of the vertex itself, but also the state of the parent and grandparent of the vertex. In our approach, we do not need retransmissions of whole sentences, paragraphs or of the whole document in order to maintain the same tree structure of the document at all sites.

Some state-based approaches for merging have been proposed. A semantic state-based approach for the merging of two different versions of a program has been proposed in (Horwitz et al. 1988). But this approach is restricted to operating on a specific programming language.

Another semantic state-based approach for merging is the flexible diff (Neuwirth et al. 1992) approach that finds and reports differences between two versions of text. The PREP writing environment which relies on the diff approach is flexible, allowing users to indicate the granularity of the changes they want to find and to control the granularity of how the changes are shown when they are reported, the choices being word, phrase, sentence or paragraph. If the user chooses a pinpointing granularity of sentence, then any word differences are shown as an old sentence deleted and a new sentence inserted. In contrast to the state-based approaches, our approach is operation-based and, as previously described, this yields a number of advantages over state-based merging.

A flexible object framework that allows the definition of the merge policy based on a particular application was adopted by the Suite collaboration system (Munson and Dewan 1994). Merging can be automatic, semi-automatic or interactive. The objects subject to collaboration are structured and therefore semantic fine-grained policies for merging can be specified. A merge matrix defines merge functions for the possible set of operations. The paper provides as an example the merge matrix for a sequence object, where the operations considered are the deletion of an element, the insertion of an element after another element in the sequence and the modification of an element. The work described in (Munson and Dewan 1994) presents how conflicts are handled between two versions of a document, but does not provide a solution for n-way merging, such as a synchronisation mechanism against a repository.

A follow-up work of (Munson and Dewan 1994) is the Sync application (Munson and Dewan 1997) that provides high-level primitives in the form of predefined classes that enable programmers to create synchronised, replicated data objects. The Sync approach for merging is based on the merge-model described in (Munson and Dewan 1994). An application of Sync for a drawing-based collaborative tool has been proposed in (Munson and Dewan 1997). Sync allows a merge of a user's change with the server, but requires that the server's change always wins in a conflict. In our approach, various policies for merging can be specified for resolving conflicts, such as executing the local changes, or the changes on the repository or let the user decide on the changes to be kept.

Our approach follows the same idea of a flexible definition and resolution of conflicts as described in (Munson and Dewan 1994, 1997). However, we implemented the flexibility for the definition and resolution of conflicts in the context of an operational transformation algorithm for maintaining the order between the semantic units of the document. We identified the semantic units of the document by their position in the tree and we provide exact algorithms for maintaining the order between the semantic units in the presence of concurrent operations. Moreover, we proposed complete merging algorithms for synchronisation against a shared repository. The approaches proposed in (Munson and Dewan 1994, 1997) do not provide any correctness criteria that should be satisfied by the proposed merging matrices when used for n-way merging. The

criteria for correctness of the transformation functions which have the same meaning as the merging matrices are well defined (Ressel et al. 1996; Suleiman et al. 1997; Sun and Ellis 1998) and can be checked (Imine et al. 2003). In our approach we can use any existing transformation functions that satisfy the correctness properties.

The CAMERA system proposed in (Lippe and van Oosterom 1992) offers an operation-based merging approach in order to support the cooperative developments that are performed on an object-oriented database. The merging approach is flexible and offers the user the possibility of selecting one of the following approaches: to impose an ordering on operations, to discard an operation involved in a conflict or to edit the result of merging. The merging algorithm takes two sequences of change operations and combines them into a single sequence, detecting both inconsistencies and conflicts. However, the algorithm is complex because a huge search space of potential merged operation sequences must be considered. However no direct application of the merging algorithm proposed in (Lippe and van Oosterom 1992) has been presented for a text-based collaborative editing system.

In GINA (Berlage and Genau 1993), the merging is performed using command histories. Document versions are represented as branches of the command history. Each branch contains the editing commands that have been executed for modifying the versions of the document. When a merge is performed, two branches are merged by applying one of the command object branches at the end of the other branch. The commands are automatically accepted when possible, but a user is given the choice of which change to keep when conflicts arise. The user may undo the operation from one branch and redo the operation from the second branch or simply not redo the operation of the second branch. GINA is shown as supporting generality with respect to the object merged because it is based on command histories. However, merging is not semantic based and there are no policies of merging defined by the user, the intervention of the user being necessary in the case of a conflict. Synchronous and decoupled modes of communication are supported as well as the transition from one mode to the other.

The IceCube (Preguica et al. 2003; Kermarrec et al. 2001) approach is a log-based reconciliation approach that is parameterised by the application semantics. The application semantics are expressed by means of the static and dynamic constraints between pairs of actions. IceCube reconciliation is viewed as an optimisation problem of scheduling a maximum number of concurrent actions given a set of constraints. The advantage of the IceCube approach is that it is looking for the combinations of concurrent operations that minimise the conflicts of reconciliation. However, the optimisation problem is NP-hard. Therefore, the resolution can be computed efficiently only by using a heuristical algorithm.

In (Molli et al. 2003), the authors propose using the operational transformation approach to define a general algorithm for synchronizing file systems and file

contents. The file systems have a hierarchical structure, however, for the merging of text documents, the authors proposed using a fixed working unit, i.e. the block unit consisting of several lines of text. The defined operations that work on the block units are: *addblock*, *deleteblock* and *moveblock*. The transformation functions are defined for each pair of operations. In our approach, we use a flexible unit for merging and due to the distributed history throughout the tree we obtain a better efficiency.

The merging approach proposed in FORCE is flexible, the semantic merging policies being separated from the syntactic merging. However, the FORCE approach has been used only for asynchronous collaborative editing on text documents conforming to a linear representation. In our approach, we applied the FORCE mechanism for merging recursively over the documents levels.

10. Conclusions and future work

We have presented a consistency maintenance algorithm relying on a tree representation of documents. The hierarchical representation of documents is a generalisation of the linear representation and, in this way, our algorithm can be seen as extending the existing OT algorithms. We have described some of the existing OT algorithms for linear documents and shown how these can be applied to hierarchical document structures, resulting in fewer transformations having to be performed.

We have also shown how our approach that can be applied to both real-time and asynchronous modes of collaboration. The algorithm applies the same basic mechanism as existing OT algorithms for synchronous or asynchronous collaboration recursively over the different document levels. The algorithm is general in that it could use any of the OT algorithms relying on a linear representation. The approach offers increased efficiency compared to existing approaches that use a linear structure for representing the document. It also offers support to track user activities to provide awareness information and handle conflicting changes that refer to certain parts of the document.

Our approach can be applied to collaboration over any hierarchically structured documents such as textual documents or XML documents. The approach could also be extended to deal with formatted text documents.

One of the future work directions would be to explore the possibility of performing undo operations in the multi-level editing approach. As the history is distributed throughout the tree, undo operations could be applied for a specific semantic unit of the tree. A previous state of a semantic unit can be restored by undoing operations belonging to the histories of that node and its child nodes.

In our current prototypes, insert operations concurrent and overlapping with a delete operation where insert operations target a subelement of the deleted node are ignored. We are currently working on the integration of the multi-level editing approach with the tombstone transformational approach (Oster et al. 2006) that

does not delete document nodes but marks them for deletion. In this way, changes performed on the deleted nodes are kept and restored in the case of an undo.

Acknowledgements

Many thanks to Gérald Oster for the valuable discussions concerning aspects of this work. We also thank the anonymous reviewers and the Associated Editor, Prasun Dewan, for their suggestions which helped improve the presentation of this paper.

References

- Allen, Larry, Gary Fernandez, Kenneth Kane, David B. Leblang, Debra Minard and John Posner (1995): ClearCase MultiSite: Supporting Geographically-Distributed Software Development. *Lecture Notes in Computer Science*, vol. 1005, pp. 194–214.
- Berlage, T. and A. Genau (1993): A Framework for Shared Applications with a Replicated Architecture. In *Proceedings of the 6th Annual ACM Symposium on User Interface Software and Technology (UIST'93)*, pp. 249–57. Atlanta, Georgia, USA: ACM Press.
- Berliner, B. (1990): CVS II: Parallelizing Software Development. In *Proceedings of the Winter 1990 USENIX Conference*, pp. 341–352. Washington, District of Columbia, USA, January.
- Collins-Sussman, B., B.W. Fitzpatrick and C.M. Pilato (2004): *Version Control with Subversion*. Cambridge, Massachusetts, USA: O'Reilly.
- Davis, A.H., C. Sun and J. Lu (2002): Generalizing Operational Transformation to the Standard General Markup Language. In *Proceedings of the 2002 ACM conference on Computer supported cooperative work (CSCW'02)*, pp. 58–67. New Orleans, Louisiana, USA: ACM Press.
- Ellis, C.A. and S.J. Gibbs (1989): Concurrency Control in Groupware Systems. *SIGMOD Record (ACM Special Interest Group on Management of Data)*, vol. 18, no. 2, pp. 399–407, June.
- Horwitz, S., J. Prins and T. Reps (1988): Integrating Non-Interfering Versions of Programs. In ACM-SIGPLAN ACM-SIGACT (eds.): *Conference Record of the 15th Annual ACM Symposium on Principles of Programming Languages (POPL'88)*, pp. 133–145. San Diego, California, USA: ACM Press, January.
- Ignat, C.-L., and M.C. Norrie (2002): Tree-Based Model Algorithm for Maintaining Consistency in Real-Time Collaborative Editing Systems. *Fourth International Workshop on Collaborative Editing, CSCW 2002, IEEE Distributed Systems online*.
- Ignat, C.-L. and M.C. Norrie (2003): Customizable Collaborative Editor Relying on treeOPT Algorithm. In K. Kuutti, E.H. Karsten, G. Fitzpatrick, P. Dourish and K. Schmidt (eds): *Proceedings of the 8th European Conference on Computer-supported Cooperative Work (ECSCW'03)*, pp. 315–334. Dordrecht, NL: Kluwer Academic Publishers 2003.
- Ignat, C.-L. and M.C. Norrie (2006a): Flexible Collaboration Over XML Documents. In *Proceedings of the International Conference on Cooperative Design, Visualization and Engineering (CDVE'06)*, pp. 267–274. Mallorca, Spain, September.
- Ignat, C.-L. and M.C. Norrie (2006b): Flexible Definition and Resolution of Conflicts Through Multi-Level Editing. In *Proceedings of the 2nd International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom'06)*. Georgia, Atlanta, USA, November.
- Ignat, C.-L. and G. Oster (2007): Flexible Reconciliation of XML Documents in Asynchronous Editing. In *Proceedings of the International Conference on Enterprise Information Systems (ICEIS'07)*. Funchal, Madeira, Portugal, June.

- Imine, A., P. Molli, G. Oster and M. Rusinowitch (2003): Proving Correctness of Transformation Functions in Real-Time Groupware. In *Proceedings of the European Conference on Computer-Supported Cooperative Work (ECSCW'03)*, pp. 277–293. Helsinki, Finland: Kluwer Academic Publishers, September.
- Ionescu, M. and I. Marsic (2003): Tree-Based Concurrency Control in Distributed Groupware. *Computer Supported Cooperative Work*, vol. 12, no. 3, pp. 329–350.
- Kermarrec, A.-M., A. Rowstron, M. Shapiro and P. Druschel (2001): The IceCube Approach to the Reconciliation of Divergent Replicas. In *Proceedings of the twentieth annual ACM symposium on Principles of distributed computing (PODC'01)*, pp. 210–218. Newport, Rhode Island, USA: ACM Press.
- Lamport, L. (1978): Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM*, vol. 21, no. 7, pp. 558–565.
- Li, D. and R. Li (2004): Preserving Operation Effects Relation in Group Editors. In *Proceedings of the 2004 ACM conference on Computer supported cooperative work (CSCW '04)*, pp. 457–466. Chicago, Illinois, USA: ACM Press.
- Li, R. and D. Li (2005): Commutativity-Based Concurrency Control in Groupware. In *Proceedings of the IEEE Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom'05)*, pp. 1–10. San Jose, California, USA: IEEE Computer Society, December.
- Li, D. and R. Li (2006): A Performance Study of Group Editing Algorithms. In *Proceedings of the 12th International Conference on Parallel and Distributed Systems (ICPADS'06)*, pp. 300–307. Washington, DC, USA: IEEE Computer Society.
- Li, D. and R. Li (2007): An Operational Transformation Algorithm and Performance Evaluation. *Journal of Computer Supported Cooperative Work* (in press).
- Lippe, E. and N. van Oosterom (1992): Operation-Based Merging. In *Proceedings of the fifth ACM SIGSOFT symposium on Software development environments*, pp. 78–87. Tyson's Corner, Virginia, USA: ACM Press.
- Molli, P., H. Skaf-Molli, G. Oster and S. Jourdain (2002): SAMS: Synchronous, Asynchronous, Multi-Synchronous Environments. In *Proceedings of the Conference on Computer-Supported Cooperative Work in Design (CSCWD'02)*, pp. 80–85. Rio de Janeiro, Brazil, September.
- Molli, P., G. Oster, H. Skaf-Molli and A. Imine (2003): Using the Transformational Approach to Build a Safe and Generic Data Synchronizer. In *Proceedings of the ACM SIGGROUP Conference on Supporting Group Work (GROUP'03)*, pp. 212–220. Sanibel Island, Florida, USA: ACM Press, November.
- Munson, J.P. and P. Dewan (1994): A Flexible Object Merging Framework. In *Proceedings of the 1994 ACM conference on Computer supported cooperative work (CSCW '94)*, pp. 231–242. Chapel Hill, North Carolina, USA: ACM Press.
- Munson, J.P. and P. Dewan (1997) Sync: A Java Framework for Mobile Collaborative Applications. *Computer*, vol. 30, pp. 59–66.
- Myers, E.W. (1986): An $O(ND)$ Difference Algorithm and its Variations. *Algorithmica*, vol. 1, pp. 251–266.
- Neuwirth, C.M., R. Chandhok, D.S. Kaufer, P. Erion, J. Morris and D. Miller (1992): Flexible Diff-Ing in a Collaborative Writing System. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW'92)*, Collaborative writing, pp. 147–154. Toronto, Ontario: ACM Press.
- Nichols, D.A., P. Curtis, M. Dixon and J. Lamping (1995) High-Latency, Low-Bandwidth Windowing in the Jupiter Collaboration System. In *Proceedings of the ACM Symposium on User Interface Software and Technology*, pp. 111–120.
- Oster, G., P. Molli, P. Urso and A. Imine (2006): Tombstone Transformation Functions for Ensuring Consistency in Collaborative Editing Systems. In *Proceedings of the Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom'06)*. Atlanta, Georgia, USA: IEEE Computer Society, November.

- Papadopoulou, S., C.-L. Ignat, G. Oster and M.C. Norrie (2006): Increasing Awareness in Collaborative Authoring Through Edit Profiling. In *Proceedings of the 2nd International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom'06)*. Georgia, Atlanta, USA.
- Prakash, A. and M.J. Knister (1994): A Framework for Undoing Actions in Collaborative Systems. *ACM Transactions on Computer-Human Interaction*, vol. 1, no. 4, pp. 295–330.
- Preguica, N., M. Shapiro and C. Matheson (2003): Semantics-Based Reconciliation for Collaborative and Mobile Environments. In *Proceedings of the Eleventh International Conference on Cooperative Information Systems (CoopIS'03)*, volume 2888 of *Lecture Notes in Computer Science*, pp. 38–55. Catania, Sicily, Italy: Springer, November.
- Ressel, M., D. Nitsche-Ruhland and R. Gunzenhäuser (1996): An Integrating, Transformation-Oriented Approach to Concurrency Control and Undo in Group Editors. In *Proceedings of the ACM Conference on Computer-Supported Cooperative Work (CSCW'96)*, pp. 288–297. Boston, Massachusetts, USA: ACM Press, November.
- Shen, H. and C. Sun (2002): Flexible Merging for Asynchronous Collaborative Systems. In *Proceeding of the Conference on Cooperative Information Systems (CoopIS'02)*, volume 2519 of *Lecture Notes in Computer Science*, pp. 304–321. Irvine, California, USA: Springer-Verlag, November.
- Suleiman, M., M. Cart and J. Ferrié (1997): Serialization of Concurrent Operations in a Distributed Collaborative Environment. In *Proceedings of the international ACM SIGGROUP conference on Supporting group work (GROUP '97)*, pp. 435–445. Phoenix, Arizona, USA: ACM Press.
- Suleiman, M., M. Cart and J. Ferrié (1998): Concurrent Operations in a Distributed and Mobile Collaborative Environment. In *Proceedings of the International Conference on Data Engineering (ICDE'98)*, pp. 36–45. Orlando, Florida, USA: IEEE Computer Society, February.
- Sun, C. (2002): Undo as Concurrent Inverse in Group Editors. *ACM Transactions on Computer-Human Interactions*, vol. 9, no. 4, pp. 309–361.
- Sun, C. and C.S. Ellis (1998): Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements. In *Proceedings of ACM CSCW'98 Conference on Computer-Supported Cooperative Work (CSCW'98)*, pp. 59–68.
- Sun, D. and C. Sun (2006): Operation Context and Context-Based Operational Transformation. In *Proceedings of the 2006 20th anniversary conference on Computer supported cooperative work (CSCW'06)*, pp. 279–288. New York, NY, USA: ACM Press.
- Sun, C., X. Jia, Y. Zhang, Y. Yang and D. Chen (1998): Achieving Convergence, Causality Preservation, and Intention Preservation in Real-Time Cooperative Editing Systems. *ACM Transactions on Computer-Human Interaction*, vol. 5, no. 1, pp. 63–108, March.
- Sun, D., S. Xia, C. Sun and D. Chen (2004): Operational Transformation for Collaborative Word Processing. In *Proceedings of the 2004 ACM conference on Computer supported cooperative work (CSCW'04)*, pp. 437–446. Chicago, Illinois, USA: ACM Press.
- Sun, C., S. Xia, D. Sun, D. Chen, H. Shen and W. Cai (2006): Transparent Adaptation of Single-User Applications for Multi-User Real-Time Collaboration. *ACM Transactions on Computer-Human Interactions*, vol. 13, no. 4, pp. 531–582.
- Tichy, W.F. (1991): RCS – A system for version control, November.
- Vidot, N., M. Cart, J. Ferrié and M. Suleiman (2000): Copies Convergence in a Distributed Real-Time Collaborative Environment. In *Proceedings of ACM Conference on Computer Supported Cooperative Work (CSCW'00)*, pp. 171–180.
- Xia, S., D. Sun, C. Sun and D. Chen (2005): Collaborative Object Grouping in Graphics Editing Systems. In *Proceedings of International Conference in Collaborative Computing: Networking, Applications and Worksharing (Collaborate Com'05)*. San Jose, California, USA.
- Zaffer, A.A., C.A. Shaffer, R.W. Ehrich and M. Perez (2001): Netedit: A Collaborative Editor. Technical report, January.